



Degree Project in Computer Science and Engineering

Second cycle, 30 credits

Explainable Artificial Intelligence for Knowledge-Based and Nested Belief-Based Strategies in Grid-Based Pursuit-Evasion Games

OSCAR ARBMAN

Explainable Artificial Intelligence for Knowledge-Based and Nested Belief-Based Strategies in Grid-Based Pursuit-Evasion Games

OSCAR ARBMAN

Degree Programme in Computer Science and Engineering
Date: June 29, 2026

Supervisor: Dillian Guruv
Examiner: Mads Dam

School of Electrical Engineering and Computer Science
Swedish title: Förklarbar Artificiell Intelligens för Kunskapsbaserade och
Hierarkiska Trosbaserade Strategier i Rutnätbaserade Jakt-Flykt Spel

Abstract

Explainable Artificial Intelligence (XAI) increases trust and transparency in AI systems, helping identify potential misalignments in a model’s reasoning process. In this thesis, agents reason based on two types of knowledge that arise from the fact that agents have imperfect information about the state: first-order knowledge, which captures what an agent can infer about the current hidden state based on its own observation history; and second-order knowledge, which also accounts for deducible first-order knowledge of other agents. Previous work on knowledge-based strategy synthesis in grid-based pursuit-evasion games with imperfect information has not provided convincing evidence that Deep Q Network (DQN) based strategies that use second-order knowledge outperform those that use only first-order knowledge. This lack of evidence raises the question of whether agents actually benefit from and utilize second-order knowledge when making decisions in this environment. Consequently, a goal of the project is to develop a framework for extracting interpretable, knowledge-based strategies from DQNs to determine whether an agent with second-order knowledge employs them in its reasoning process within this environment.

The environment this thesis investigates is a (15×15) grid-based pursuit-evasion game with two pursuers and one evader with imperfect information. However, because the current second-order knowledge-based update function produces too large representations, this thesis introduces a training pipeline to approximate second-order knowledge with a neural network. To distinguish it from the original knowledge-based update function, we refer to the approximation as the belief-based update model. The thesis then presents a symbolic representation that enables the use of an imitation learning method called INTERPRETER, which generates an interpretable decision-tree strategy in Python code that mimics the oracle DQN’s actions. The thesis’s three key findings were: firstly, that the decision tree strategy using second-order beliefs achieved the highest reward of any investigated strategy, with the lowest average steps to a terminal state of 18.976, and a win rate of 99.991%; and secondly, that the strategy bases its decisions partly on these second-order beliefs. Moreover, thirdly, an interpretable, completely formulaic from-observations-to-actions, first-order, knowledge-based strategy that was almost surely winning, losing only 2 out of 1,000,000 games, with an average number of steps to a terminal state of 20.031, comparatively a naive perfect-information strategy, that always minimizes the distance to the evader achieved a 15.742 average steps to a terminal state, with a 100% win rate.

Keywords

Artificial Intelligence, Explainable Artificial Intelligence, Reinforcement Learning, Imperfect Information, Pursuit-Evasion Games, Grid-Based, Knowledge-Based Strategies, Deep Q-Networks, Convolutional Neural Networks, Long Short-Term Memory, Decision Trees, INTERPRETER, Knowledge-Based Update, Belief-Based Update

Sammanfattning

Förklarbar artificiell intelligens (XAI) ökar förtroendet och förståelsen för AI-nätverk, vilket kan hjälpa till att identifiera potentiella fel i hur neurala nätverk resonerar. I denna avhandling resonerar agenter utifrån två typer av kunskap som uppstår eftersom agenterna har ofullständig information om spelets nuvarande tillstånd. Första ordningens kunskaper, fångar slutsatserna en agent kan dra om det nuvarande speltillståndet baserat på agentens observationshistoria och andra ordningens kunskaper, fångar också de härledbara första ordningens kunskaperna om de andra agenterna. Tidigare strategisyntes arbeten inom kunskapsbaserade förföljelse-undvikande spel på en rutnät baserad miljö med ofullständig information har inte funnit övertygande bevis för att djupa Q-nätverks (DQN)-baserade strategier som har tillgång till andra ordningens kunskaper är bättre än de som endast har första ordningens kunskaper. Denna brist på bevis väcker frågan om agenter faktiskt drar nytta av och använder andra ordningens kunskaper när de fattar beslut i denna miljö. Därmed är ett mål med denna avhandling att utveckla ett ramverk för att extrahera tolkningsbara, kunskapsbaserade strategier från DQN för att avgöra om en agent med andra ordningens kunskaper använder dem i sina resonemang i denna miljö.

Miljön som denna avhandling undersöker är ett (15×15) rutnätbaserat förföljande-undvikande-spel med 2 förföljare och 1 undvikare, där informationen är ofullständig. Eftersom den nuvarande andra ordningens kunskapsbaserade uppdateringsfunktionen producerar för stora representationer, introducerar denna avhandling en träningsprocess för att approximera andra ordningens kunskaper med ett neuralt nätverk. För att skilja denna approximation från den ursprungliga kunskapsbaserade uppdateringsfunktionen refererar vi till approximationen som den trosbaserade uppdaterings modellen. Avhandlingen presenterar sedan en symbolisk representation som möjliggör användningen av en imitation-inlärningsmetod som kallas INTERPRETER, vilket genererar en tolkningsbar beslutsträdstrategi i Python-kod som efterliknar orakel-DQNs handlingar. Avhandlingens tre viktigaste resultat var: för det första, att beslutsträdstrategin som använder andra ordningens trosuppfattningar uppnådde den högsta belöningen av alla undersökta strategier, med den lägsta genomsnittliga mängden steg till ett terminalt tillstånd på 18,976 och en vinstfrekvens på 99,991%; och för det andra, att strategin delvis baserar sina beslut på sina andra ordningens trosuppfattningar. Dessutom, för det tredje, hittades en komplett formell första ordningens kunskapsstrategi från observation till handling, som var nästan garanterat vinnande, som

enbart förlorade 2 av 1,000,000 matcher, med ett genomsnittligt antal steg till ett terminalt tillstånd på 20,031. Jämförelsevis hade en naiv avståndsförminskande strategi med perfekt information ett genomsnittligt antal steg till ett terminalt tillstånd på 15,742 och en vinstfrekvens på 100%.

Nyckelord

Artificiell Intelligens, Förklarbar Artificiell Intelligens, Förstärkningsinlärning, Imperfekt Information, Förföljelse-Undvikande Spel, Rutnäts Baserad, Kunskapsbaserade Strategier, Djupa Q-Nätverk, Faltningsnätverk, Långt Korttidsminne, Beslutsträd, INTERPRETER, Kunskapsbaserad Uppdatering, Trosbaserad Uppdatering

Acknowledgments

I would like to thank **Dilian Gurov** for the opportunity to work on this project and for his mentorship. I also acknowledge that this work was written with the help of **Grammarly** to improve spelling, grammar, punctuation, and overall sentence flow. As of late, it seems **Grammarly** has begun using generative AI to improve its text-change suggestions, which makes it somewhat of a gray area. The only outside-sourced graphic in this thesis is from [\[1\]](#), which is my own, and Olof Lilliestierna's bachelor's thesis.

Stockholm, June 2026
Oscar Arbman

Contents

1	Introduction	1
1.1	Background	2
1.2	Problem	4
1.3	Purpose	5
1.4	Goals	5
1.5	Research Methodology	5
1.6	Delimitations	6
1.7	Structure of the thesis	6
2	Background	7
2.1	Knowledge-Based Strategies in Grid-Based Pursuit-Evasion Games	7
2.1.1	Grid-Based Pursuit-Evasion Games of Imperfect Information	7
2.1.2	Knowledge-Based Strategies	8
2.1.3	Related Work: Knowledge-Based Strategies in Grid-Based Pursuit-Evasion Games	9
2.2	Reinforcement Learning and Neural Nets	9
2.2.1	Markov Decision Process	9
2.2.2	Q-Learning	10
2.2.3	Deep Q-Networks	11
2.2.4	Multilayer Perceptron	11
2.2.5	Convolutional Neural Networks	12
2.2.6	Deep Q-Network (DQN) Training Problems and Solutions	13
2.2.6.1	Experience Replay	13
2.2.6.2	A Separate Target Network	13
2.2.6.3	Double DQN	14
2.2.7	Partially Observable Markov Decision Process	14

2.2.8	Deep Recurrent Q-Learning for Partially Observable Markov Decision Process (POMDP)s	15
2.2.9	Recurrent Neural Network (Recurrent Neural Network (RNN))	15
2.2.9.1	Backpropagation Through Time	16
2.2.9.2	Vanishing or Exploding gradients	16
2.2.10	Long Short Term Memory (Long Short-Term Memory (LSTM))	16
2.2.10.1	Modeling Missing Input Data with LSTMs	17
2.2.11	Related Work: Reinforcement Learning and Neural Nets	18
2.2.11.1	Reducing Model Overfitting	18
2.2.11.2	Particle Filters	18
2.3	Explainable Artificial Intelligence	19
2.3.1	Distilling a Neural Network Into a Soft Decision Tree	20
2.3.2	INterpretable Editable tRee Programs for ReinforceE-mentT lEaRning (INTERPRETER)	20
2.3.2.1	The Input Data	21
2.3.2.2	DAGGER	21
2.3.2.3	Q-DAGGER	22
2.3.2.4	INTERPRETER's Oracle Data	22
2.3.2.5	Oblique Decision Trees	22
2.3.3	Related Work: Explainable Artificial Intelligence	23
2.3.3.1	Feature-Importance Explanations	23
2.3.3.2	A Unified Approach to Interpreting Model Predictions	24
2.3.3.3	SkillTree an Abstract Decision Tree (DT)	24
2.3.3.4	Symbolic Representations	24
2.3.3.5	Symbolic Predicate Invention	25
2.3.3.6	Extracting a Mathematical Explanation	25
2.3.3.7	Learning Finite State Representations of Recurrent Policy Networks	26
3	Methodology	29
3.1	Overview of the Contributions	29
3.2	Benchmark Strategies	30
3.2.1	The First-Order Knowledge-Based Update (KBU)-DQN Strategy	30
3.2.2	The Naive Perfect-Information Strategy	31
3.3	Data Collection and Sample Size	31

3.4	Statistical Significance of the Results	31
3.5	Model Selection of Decision Trees	32
4	A Framework for Explainable Knowledge-Based Strategies	33
4.1	Architectural Overview	33
4.2	The Pursuit-Evasion Games (PEG) Environment	34
4.3	Notes on Agent Independence	35
4.4	Knowledge-Based Update Function	36
4.5	Belief-Based Update Model	37
4.6	First-Order Belief-Based Update	37
4.7	From Knowledge/Belief to Action	39
4.8	Second-Order Belief-Based Update	40
4.8.1	Notes on Second-Order Particle-Filter Beliefs	41
4.9	Extracting an Interpretable Strategy	42
4.10	A Symbolic Representation	43
4.11	INTERPRETER	43
5	Results and Analysis	45
5.1	Performance Comparison of Oracle DQNs	45
5.2	Distilled Decision Tree Metrics by Size	46
5.3	Performance Comparison of Distilled Decision Tree Strategies	47
5.4	Inspecting the Decision Trees	47
5.4.1	First-Order KBU-DT	48
5.4.2	First-Order Belief-Based Update (BBU)-DT	48
5.4.3	Second-Order BBU-DT	49
5.5	Reliability Analysis	49
5.5.1	Statistical Analysis of Capture Rates	50
5.5.1.1	Old Model vs. New Models	50
5.5.1.2	All Models vs. The Naive-Perfect Informa- tion Strategy	50
5.5.1.3	Comparison of the New Models	51
5.5.2	Statistical Analysis of Steps to Terminal State	51
5.5.2.1	Old Model vs. New Models	51
5.5.2.2	All Models vs. The Naive-Perfect Informa- tion Strategy	52
5.5.2.3	Comparison of the New Models	52
6	Discussion	53
6.1	The Oracle DQNs	53
6.2	The Second-Order Architecture	54

6.3	Distilled Decision Trees	55
6.4	Limitations	57
6.4.1	Ethics and Sustainability	57
6.5	Reflections	57
7	Conclusions and Future Work	59
7.1	Future work	60
7.1.1	Improving Upon the Current Implementation	60
7.1.2	Analysing the DTs	60
7.1.3	XAI for the BBU model	61
7.1.4	A Concise Second-Order KBU-function	61
7.1.5	Higher Orders Strategies	61
7.1.6	Implementing the Particle-Filter Second-Order Belief Update	62
7.1.7	Different Symbolic Representations	62
	References	63
A	Supporting materials	67

List of Figures

1.1	A grid-based PEG where blue, yellow, and grey represent pursuers, evaders, and walls, where red highlights a pursuer, taken from [1].	3
2.1	A grid-based PEG where blue, yellow, and grey represent pursuers, evaders, and walls, where red highlights a pursuer, taken from [1].	7
4.1	Depiction of the pursuer's current observation in the game; note that neither agent has observed the other entering or exiting its field of view.	34
4.2	First-Order Knowledge: Note that an agent's depicted knowledge is saved before acting, so when another agent has acted afterward, the agent's depicted knowledge state will not be based on the current game state.	36
4.3	First-Order Beliefs: Note that an agent's depicted beliefs are saved before acting, so when another agent has acted afterward, the agent's depicted belief state will not be based on the current game state.	38
4.4	Second-Order Beliefs: Note that an agent's depicted beliefs are saved before acting, so when another agent has acted afterward, the agent's depicted belief state will not be based on the current game state.	41
4.5	A very small interpretable first-order, KBU-DT strategy with a max leaf node count of 8, pruned down to 5. Full code available at: GitHub.	44

5.1	Relationships between DT size and (action agreement, capture rate, and pursuer steps to a terminal state) for distilled DT after 40,000 validation simulations. The red circle marks the selected DT for evaluation.	46
5.2	A code snippet from the first-order KBU-DT.	48
5.3	A code snippet from the first-order BBU-DT.	48
5.4	A code snippet from the second-order BBU-DT.	49

List of Tables

5.1	DQN results after 1 million validation simulations; all models were simulated on the same held-out-evaluation set of seeds	45
5.2	Distilled DT results after 1 million validation simulations; all models were simulated on the same held-out-evaluation set of seeds	47
5.3	Pairwise binomial McNemar test for win rate comparing the old model against the new models.	50
5.4	Pairwise binomial McNemar test for win rate comparing all models against the naive perfect information strategy.	50
5.5	Pairwise binomial McNemar test for win rate comparing all new models.	51
5.6	Pairwise t-test for average steps to a terminal state comparing the old model vs the new models.	51
5.7	Pairwise t-test for average steps to a terminal state comparing all models against the naive perfect information strategy.	52
5.8	Pairwise t-test for average steps to a terminal state comparing all new models.	52
6.1	DQN results with different rewards, after 1,000,000 evaluation simulations, on the same held-out set of seeds.	53
6.2	Limiting the DTs' feature representation, disallowing it to subtract any symbolic feature except directional probabilities, i.e., the agent's own position and the time left scalar, is only allowed to be compared standalone without subtraction.	56
7.1	The most interesting results	60

List of acronyms and abbreviations

Adam	Adaptive Moment Estimation
AI	Artificial Intelligence
BBU	Belief-Based Update
BPTT	Backpropagation Through Time
CNN	Convolutional Neural Network
DAGGER	Dataset Aggregation
DNN	Deep Neural Network
DQN	Deep Q-Network
DRQN	Deep Recurrent Q-Network
DT	Decision Tree
INTERPRETER	INTErpretable Editable tRee Programs for ReinforcEmenT lEaRning
KBU	Knowledge-Based Update
LSTM	Long Short-Term Memory
MAGIIAN	Multi-Agent Game with Imperfect Information Against Nature
MDP	Markov Decision Process
MLP	Multilayer Perceptron
MNM	Model Non-Manipulability
MNO	Model Non-Observability
PEG	Pursuit-Evasion Games
PF-RNNs	Particle Filter Recurrent Neural Networks
POMDP	Partially Observable Markov Decision Process
ReLU	Rectified Linear Unit
RL	Reinforcement Learning
RNN	Recurrent Neural Network
XAI	Explainable Artificial Intelligence

Chapter 1

Introduction

Explainable Artificial Intelligence (XAI) increases trust and transparency in **Artificial Intelligence (AI)** systems, often by probing their opaque, black-box nature to uncover behavior and help identify potential misalignments in a model’s reasoning. As illustrated by [2], feature-importance explanations (i.e., a type of **XAI**) can uncover cases where a wolf-versus-husky neural network image classifier achieves seemingly strong performance by learning to rely on the typical snowy background characteristics of the images with wolves to determine if the animal is a wolf or a husky, rather than the animals’ characteristics themselves, leading to poor generalization when snow is no longer a reliable cue, where **XAI** helps understand why this failure occurs.

Therefore, to better understand why previous work, such as [3], has not found convincing evidence that second-order knowledge benefits agents in this environment, this thesis investigated state-of-the-art **XAI** methods. To create a framework for extracting human-understandable knowledge-based strategies on grid-based **Pursuit-Evasion Games (PEG)** to investigate whether second-order knowledge is beneficial and used, and to learn which knowledge states cause an agent to perform certain actions, such that a human could reproduce them, and allow future work to verify these strategies.

However, **XAI** methods are often limited to a small input feature space. This limitation means that, without altering the old second-order **Knowledge-Based Update (KBU)** function, it becomes infeasible to use for an **XAI** solution, even for a game with just two pursuers and one evader on a small (5×5) grid. This infeasibility is because modeling a single pursuer teammate’s knowledge at a given time step requires considering all possible positions the teammate might be in and all possible observation histories for each position up to that point. Moreover, we generally do not know what the teammate

observes unless the pursuers are standing on top of each other, or we know both the teammate’s and the evader’s positions to such an extent that we can deduce what the teammate will observe; thus, the second-order knowledge representation grows rapidly with the number of possible states and agents.

Therefore, this thesis introduces a training pipeline to approximate second-order knowledge with a neural network. To distinguish it from the original **KBU** function, we refer to the neural network approximation as the **Belief-Based Update (BBU)** model. The pipeline alternates between training and freezing the **BBU** model and the belief-to-action mapping **Deep Q-Network (DQN)** model of different knowledge orders. To create first-order beliefs (i.e., beliefs about the current state), this thesis uses an **Long Short-Term Memory (LSTM)**-based architecture to generate two probability maps over the possible positions of the evader and teammate on the grid. Moreover, to create second-order beliefs (i.e., beliefs about teammates’ beliefs), this thesis uses a second **LSTM** that takes input from the main agent’s observations and from the first **LSTM**, producing two additional probability maps representing beliefs about beliefs.

Rather than using two separate **LSTMs**, they are stacked in the second-order **BBU** model to encourage a hierarchical representation: the lower **LSTM** learns a representation of the current hidden state, which the upper **LSTM** can condition on when modeling second-order beliefs. The thesis then presents a symbolic representation that compresses the probability grids into weighted probability masses associated with each action. This symbolic representation reduces the second-order belief representation from 900 to 16 belief features for a two-pursuer, one-evader (15×15) grid. This symbolic representation of the belief grids (combined with the agent’s location and a time-left scalar) enables the use of an imitation learning method called **INTERpretable Editable tRee Programs for ReinforcEmenT IEaRning (INTERPRETER)**, which generates an interpretable **Decision Tree (DT)** strategy in Python code that mimics the oracle **DQN**’s actions.

1.1 Background

The grid-based environment is a 2D matrix (as shown in Figure 1.1), and Pursuit-Evasion is a two-team setting in which pursuers try to capture evaders in as few actions as possible. In this setting, the evader is represented as nature, selecting actions at random, making it impossible for a pursuer’s strategy to depend on an evader’s strategy to guarantee a victory. Note that agents trained with **DQN** are implicitly encouraged to learn the other agents’ behavioral

patterns, as this is beneficial for optimizing capture in a shared environment.

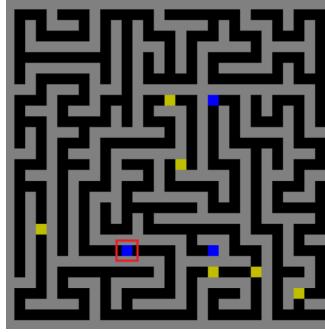


Figure 1.1: A grid-based **PEG** where blue, yellow, and grey represent pursuers, evaders, and walls, where red highlights a pursuer, taken from [1].

Knowledge in this setting stems from the fact that the games we study have imperfect information [4], in which agents can observe only positions in the same row or column as themselves. A knowledge-based strategy chooses actions based on its knowledge, and different levels of orders help it estimate which action will be most beneficial for achieving its goals [4]. A first-order agent estimates the current state of affairs based on the current observations and the sequence of all previous observations and actions; i.e., the set of current possible states is a subset of the set of states consistent with the observations. A second-order agent also incorporates an estimate of other agents' first-order knowledge into their knowledge. Previous work by [3] used a set-based knowledge representation in which uncertainty about unseen agents is propagated by expanding an agent's possible locations to those reachable via its valid actions. For any currently observable agent and all observable empty cells, position uncertainty becomes zero. However, their set-based representation [3] explodes in size when accounting for nested beliefs, and it doesn't capture that some possible states may be less likely than others.

Paper [5] introduced **Deep Recurrent Q-Network (DRQN)**, where an **Convolutional Neural Network (CNN)** extracts features from the observations, and agents' beliefs are maintained by an **LSTM**. An **LSTM** is a specialized form of **Recurrent Neural Network (RNN)**, which can, for example, learn and be used to classify how likely different states are to be true based on a series of observations. It logically follows that an **LSTM** can also be trained to learn nested beliefs. To extract an interpretable learned **DQN** policy, this thesis uses the method introduced in [6] called **INTERPRETER**, which distills a deep **Reinforcement Learning (RL)** policy into compact, editable

DT policies, enabling inspection, modification, and testing of the policy in a human-understandable format. Based on the literature study, **INTERPRETER** currently appears to be the best and most applicable method for robust, accurate **RL** policy extraction for our environment. However, **INTERPRETER** cannot handle a large input space, meaning the old knowledge representation of [3] will not suffice, as the second-order **KBU** scales poorly, producing hundreds of outputs for grid sizes even as small as a (5×5) , which would render the **INTERPRETER** method infeasible.

1.2 Problem

Most state-of-the-art policy-extraction **XAI** methods scale poorly with respect to the size of the feature input space. For example, paper [6] states that for their environments with up to 200 state features, their “*oblique tree extraction gives Out-of-Memory errors*”, leading the authors to remove constant or idle state features [6]. Additionally, paper [7] states that “*as one descends the tree, the expected fraction of the data that each node sees in any given training batch decreases exponentially*”. Therefore, given **XAI**’s limitation in requiring a small input space that still preserves all essential features, the research questions become:

RQ1: How can a knowledge-based **PEG-RL** strategy be designed such that it is possible to extract a faithful and interpretable policy that explains, for any specific knowledge state, how the black-box decided upon which action to take?

RQ2: Can our knowledge-based **PEG-RL** algorithm provide a second-order strategy that shows a significant improvement over its corresponding first-order strategy? How do these strategies compare to a naive perfect-information, distance-minimization strategy?

RQ3: With what accuracy will the extracted policy give the identical action, and since some actions might lead to equivalent outcomes, how will the performance also differ between the extracted and original policies? How will the extracted strategies differ between first- and second-order in relation to performance, and will the second-order explanation showcase second-order reasoning?

1.3 Purpose

The purpose of the thesis is to develop a framework for extracting knowledge-based strategies from a **DQN** so that a human can understand and follow the knowledge-to-action mapping. Achieving this goal will help future work on knowledge-based strategies in grid-based pursuit-evasion games to identify misalignment in how their agents make decisions. An ethical dilemma is that pursuit-evasion strategies can, for example, be applied to drone warfare; furthermore, the resulting interpretable solution will be computationally far lighter than a **DQN**, making it more applicable to a low-cost drone. However, the resulting strategies are likely to be too simple to deploy on a drone. From a sustainability perspective, the fact that interpretable solutions are lighter makes them more resource-efficient, helping boost sustainability.

1.4 Goals

A goal of the project is to find a second-order strategy that, with statistical significance, outperforms the first-order strategies. A follow-up subgoal is for the explainable strategy to showcase that the second-order strategy actually utilizes its beliefs about other agents' beliefs.

1.5 Research Methodology

The project consisted of a broad literature review, first to understand which state-of-the-art methods exist for extracting a human-understandable strategy, and to assess their limitations. Since most state-of-the-art policy-extraction **XAI** methods scale poorly with the size of the feature input space [7], this thesis investigated methods for a second-order **KBU** function that could avoid a large output space. The thesis first investigated a particle filter approach [8, 9, 10, 11]. However, due to time constraints and the lackluster performance of the implemented particle filter, the implementation was shifted to instead utilize a lower-level **LSTM** for first-order beliefs and an upper-level **LSTM** for second-order beliefs. Although these belief grids are still too large to use the **INTERPRETER** policy extraction method [6]. Therefore, this thesis examined symbolic representations, such as belief space partitioning [12]; however, because these representations did not seem perfectly applicable, the thesis instead developed its own symbolic representation that compresses each probability grid into weighted probability masses for each direction.

The reliability of the results is assessed pairwise for each model using a binomial McNemar test on each strategy's win rate, and a t-test on the average number of steps to a terminal state, with all models tested on the same 1,000,000 simulations on a held-out evaluation set of seeds. The validity of the results was assessed by comparing the new models against a **DQN** utilizing the first-order **KBU** function (with slight modifications) from [3] and a naive perfect-information strategy that always minimizes the distance to the evader.

1.6 Delimitations

This thesis will not develop an interpretable solution for how the **LSTMs** map the history of observations to beliefs for two reasons. Firstly, the goal of the thesis is to create a framework for explaining knowledge-to-action, since there is already a formulaic method for mapping a history of observations to knowledge. Although it is not feasible to extract an interpretable solution from the current second-order **KBU** function due to its large output space, the first-order **KBU** function is feasible with slight modifications. Second, no robust methods were found in the literature for extracting explainable solutions from **LSTMs**. Furthermore, this thesis will not develop a formulaic second-order **KBU** function, as this is out of scope. If further work develops one, they could still utilize this extraction method for knowledge-based strategies. Moreover, this thesis will not perform a formal analysis of how the interpretable solution works, but will make observations about the decisions, measure its performance, and compare it against the oracle **DQNs**.

1.7 Structure of the thesis

Chapter 2 presents relevant information, beginning with an introduction to knowledge-based strategies in grid-based **PEG**, followed by an introduction to **RL**, **DQNs**, and **LSTMs**, and, lastly, an overview of different **XAI** methods. Chapter 3 first covers the thesis contributions and the methods used to examine and validate the results of these contributions. Chapter 4 gives a deep dive into the developed framework and all major design decisions. Chapter 5 presents all results from the contributions and analyzes the model's performance using statistical analysis of average steps to a terminal state and win rate. Chapter 6 presents the interpretation of the results, potential ethical and sustainability considerations, and reflections on the framework. Finally, Chapter 7 presents the major conclusions and future work.

Chapter 2

Background

2.1 Knowledge-Based Strategies in Grid-Based Pursuit-Evasion Games

This section outlines the fundamental research setting. It begins by describing grid-based pursuit-evasion games with imperfect information, then discusses knowledge-based strategies, and concludes by describing relevant prior work.

2.1.1 Grid-Based Pursuit-Evasion Games of Imperfect Information

In this setting, grid-based refers to an environment represented as a 2D matrix (shown in Figure 2.1). Moreover, Pursuit-Evasion is a two-team setting in which pursuers try to capture evaders in as few actions as possible.

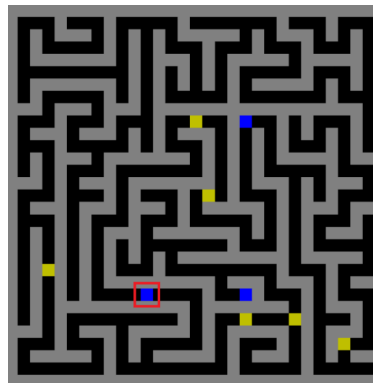


Figure 2.1: A grid-based **PEG** where blue, yellow, and grey represent pursuers, evaders, and walls, where red highlights a pursuer, taken from [1].

This setting is modeled as a **Multi-Agent Game with Imperfect Information Against Nature (MAGIIAN)**. MAGIIAN models discrete, finite, and known environments, where imperfect information causes some game states to be indistinguishable for some agents, where all agents play as a team against nature, but the agents cannot communicate with each other. Depending on the game scenario, agents may or may not observe other agents' actions, and know their strategies [4]. In this context, the evader is represented as nature, selecting actions at random, making it impossible for a pursuer's strategy to depend on any particular evader's strategy to guarantee victory.

2.1.2 Knowledge-Based Strategies

In this setting, knowledge arises from the imperfect information available to the agents [4]. A knowledge-based strategy consists of a knowledge representation, a knowledge update function, and a knowledge-to-action function. Different orders of knowledge help an agent estimate the hidden state and the corresponding best action [4].

- **0-order:** At time step t a 0-order agent estimates the current state of affairs based only on the current observations o_t .
- **1-order:** At time step t a 1-order agent estimates the current state of affairs based on the current observations o_t and the sequence of all previous observations and actions $h_{t-1} = (o_{1:t-1}, a_{1:t-1}^{self}, a_{1:t-1}^{obs})$, i.e., the set of current possible states is a subset of the set of states coherent with the observation o_t .
- **2-order:** A 2-order agent utilizes its 1-order knowledge combined with an estimate of the other agents' 1-order knowledge from observed behavior or negative information. For example, based on my knowledge of the current state, I believe you believe "Y" and are therefore likely to do "X"; therefore, I will choose to perform "Z".
- **(K+1)-order:** 1-order knowledge as the fundamental base, combined with K-nested reasoning, of the other agents' beliefs.

This thesis focuses on first- and second-order knowledge, since zero-order knowledge intuitively discards important information and has been shown to perform poorly in this environment by numerous prior works [1, 3]. However, prior work has not demonstrated that second-order knowledge is beneficial. Moreover, future work could likely extend the proposed neural network approximation of knowledge to model higher-order beliefs.

2.1.3 Related Work: Knowledge-Based Strategies in Grid-Based Pursuit-Evasion Games

Paper [3] investigated the performance of knowledge-to-action strategies created by a **DQN** (for details about **DQNs**, see Section 2.2.2 and 2.2.3). Paper [3] found no significant difference in the performance between their first- and second-order strategies. Paper [3]’s first-order knowledge representation consists of an overlapping map where each state is symbolized by a different integer (e.g., an integer for unknown, possible evader state, possible teammate state, and wall). Their **KBU** function was set-based; for the first-order strategy, uncertainty for unseen agents is propagated by expanding an agent’s possible locations to those reachable via its valid actions. For any currently observable agent and all observable empty cells, position uncertainty becomes zero. Then, for their second-order knowledge-based strategy, paper [3] updates a corresponding first-order matrix for every possible location of another pursuer by running a first-order update. Therefore, the input to the **DQN** becomes quite large for a second-order strategy, since given a (5×5) matrix with 4 walls, the number of possible cells they need to model for the first-order is $(25 - 4 = 21)$ plus the position of the agent, however for the second-order its $(25 - 4) + (25 - 4)^2 = 462$ possible cells, and 2 more inputs for the agents position, meaning the total number of inputs are 464 for a relatively small 5×5 grid with 4 walls. Moreover, this second-order **KBU** function does not account for uncertainty in the teammate’s entire observation history, so its legitimacy is called into question, supported by the fact that second-order knowledge-based strategies did not outperform first-order strategies.

2.2 Reinforcement Learning and Neural Nets

2.2.1 Markov Decision Process

A typical **Markov Decision Process (MDP)** models a single agent who repeatedly chooses actions based on perfect information about the complete current state in a stochastic environment [5]. The **MDP** is defined as a tuple $\langle \mathcal{S}, \mathcal{A}, T, R, \gamma \rangle$, where $\mathcal{S}$ is a finite set of states, $\mathcal{A}$ is a finite set of actions, $T(s, a, s')$ is the transition function that gives $P(s'|s, a)$, i.e., the probability of transitioning to state s' after taking action $a \in \mathcal{A}$ in state $s \in \mathcal{S}$. Notably, in our setting, action execution is deterministic: e.g., if the agent chooses to walk up, it will move up with certainty (assuming it is within the matrix bounds). However, the agent’s information about the next state is not determined solely

by the action, since the agent does not have perfect information and therefore will often not know what is observable after acting. The reward function is defined as $R(s, a)$ for taking action $a \in \mathcal{A}$ in state $s \in \mathcal{S}$, and γ is the discount factor $\gamma \in (0, 1)$ that weighs potential future rewards against immediate rewards, e.g., a bird in the hand is worth two in the bush.

A perfectly optimal policy π^* defines the agent's state-action pairs (i.e., $\pi^*(s) = a$) such that it obtains the highest possible expected reward [13]. The type of problem this thesis investigates is finite-horizon, meaning there is a limited number of time steps before the game terminates; therefore, the state-action mapping policy π_t can differ across the t time steps, a characteristic known as a nonstationary policy [13]. The value of a nonstationary policy at the terminal time step is given by $V^{\pi_{terminal}}(s) = R(s, \pi_{terminal}(s))$, and the value from executing a sequence of t time steps by a given policy is expressed by the Bellman Eq. (2.1). Furthermore, we know that there exists a stationary policy, i.e., a policy with an unlimited number of time steps before termination, that achieves an optimal expected reward for every initial state, expressed by the Bellman optimality Eq. (2.2) [13, 14].

$$V^{\pi_t}(s) = R(s, \pi_t(s)) + \gamma \sum_{s'} P(s'|s, \pi_t(s)) V^{\pi_{t-1}}(s') \quad (2.1)$$

$$V^{\pi^*}(s) = \max_a \{R(s, a) + \gamma \sum_{s'} P(s'|s, a) V^{\pi^*}(s')\} \quad (2.2)$$

2.2.2 Q-Learning

Q-learning is a model-free **RL** algorithm that learns the optimal stationary state-action mapping policy (Eq. 2.2) for **MDPs**, under certain assumptions. Model-free here refers to updating the $Q(s, a)$ values, i.e., the expected value of action a in state s , through trial and error without explicitly learning the **MDPs** transition and reward function $T(s, a, s')$ and $R(s, a)$ respectively [14]. To achieve an optimal state-action mapping, the Q-learner must find $\max_a (Q^*(s, a)) = V^{\pi^*}(s)$. Upon finding this, an agent can act optimally during execution by selecting actions greedily based on the maximum Q^* values. The Q values are iteratively updated during training. At iteration i , the Q-table is updated according to Eq. (2.3), i.e., the Q-values are updated based on the current reward for performing action a_i in that state s_i , and an estimate of how good the next state s_{i+1} is [14].

$$Q_i(s, a) = \begin{cases} (1 - \alpha_i) Q_{i-1}(s, a) + \alpha_i[r_i + \gamma V_{i-1}(s_{i+1})] & \text{if } s = s_i \text{ and } a = a_i, \\ Q_{i-1}(s, a) & \text{otherwise.} \end{cases} \quad (2.3)$$

Where $(V_{i-1}(s_{i+1}) = \max_a \{Q_{i-1}(s_{i+1}, a)\})$, and α is the learning rate that balances new versus prior experiences, with a high or low α , respectively. Paper [14] proves that if every state-action pair is tried infinitely often, with bounded rewards (i.e., $|r_n| \leq R$) and an appropriate decaying learning rate (2.4), tabular Q-learning will always converge to the optimal stationary Q^* values [14]. Where $\alpha_i(s, a)$, represents the learning rate during the i^{th} time we select action a in state s .

$$\sum_{i=1}^{\infty} \alpha_i(s, a) = \infty, \quad \sum_{i=1}^{\infty} (\alpha_i(s, a))^2 < \infty, \quad \forall s, a, \quad (2.4)$$

2.2.3 Deep Q-Networks

Paper [15] tackles the problem of creating a state-action policy for Atari games; although paper [15] models their environment as an **MDP**, the agent only has access to information on the game screen, i.e., not the complete current state. Paper [15] defines the current state as the full sequence of past game screen observations and actions; however, for the implementation, they fix the length to the 4 most recent frames, creating inputs of size $(84 \times 84 \times 4)$. Paper [15] explains that using Eq. (2.2) as a value iteration approach to find an optimal state-action mapping is infeasible for large problems such as this, since each sequence of observations and actions will be treated separately. Therefore, paper [15] introduces **DQN** denoted as $(Q(s, a; \theta))$, to approximate the values of each state-action pair, i.e., the $Q(s, a)$ values. Their **DQN** architecture consists of a **CNN**, that takes in the 4 stacked frames as inputs, and where the convolutional layers perform feature extraction and are followed by two fully connected layers that, for intuition, can be viewed as an **Multilayer Perceptron (MLP)** head that outputs a Q-value for each action at each state, and is trained end-to-end.

2.2.4 Multilayer Perceptron

An **MLP** is one of the most fundamental feedforward neural networks; its goal is to approximate some function (in our case, we want the **MLP** to map the **CNNs** extracted features to Q-values), consisting of an input layer, an

output layer, and one or more fully connected hidden layers of neurons [16]. Feedforward means that information flows strictly in one direction from input to output. In each layer (except the input layer), neurons compute weighted sums of the previous layer's outputs, with an added bias. Unlike a simple perceptron, the neurons in these hidden layers use nonlinear differentiable activation functions (without partial differentiability, it is not possible to propagate gradient-based error signals backward through the layers using backpropagation), such as a **Rectified Linear Unit (ReLU)**, i.e., $f(x) = \max(0, x)$, causing a cascade of weighted summations between the layers. Without nonlinear activation functions, stacking multiple linear layers would still amount to a single linear layer transformation, offering no real advantage [16]. In the case of [15], if we count the **CNN** feature extraction as the input layer, the **MLP**, then consists of 1 hidden layer consisting of 512 neurons with **ReLU** activation functions, followed by the output layer, with 1 neuron corresponding to each possible action with a linear activation function, that outputs the corresponding Q-value for performing that action in the current state.

2.2.5 Convolutional Neural Networks

A **CNN** is designed for grid-like input data and is famously used for tasks such as image classification [16]. For example, paper [2] used a type of **CNN** for their Husky vs. Wolf image classifier, which, problematically, learned to classify animals as wolves based on their typical snowy background. A **CNN** typically handles multiple channels for different feature mappings; e.g., a **CNN** for image classification typically takes 3 input channels of integer values on the grid representing the different pixel colors in red, green, and blue.

In the case of [15], the **CNN** takes in 4 images of size 84^2 pixels, representing the 4 past observed frames; since they grayscale the images, they only need 1 channel per image, resulting in input of size $(84 \times 84 \times 4)$, a convolutional layer applies a specified number of learnable filters, in the first layer they use 32 filters, with a chosen kernel size of $(8 \times 8 \times 4)$ meaning each filter performs convolutional operations on 8^2 pixels with a depth of 4, i.e., the number of channels, which can be viewed as 3D mini-grids of the input, these kernels perform a sliding-window convolution on the 2D pixel grids, each filter produces one output feature map of the input channels, in this case 32 of them. The stride defines how far the filter moves at each iteration; e.g., a stride of 4 moves the sliding window 4 positions, thereby substantially reducing the size of the resulting output. Since the kernels in the convolutional layers perform

a sliding window over the pixels, we can add zero-padding around the input grid to increase the number of times edge pixels are included in the kernel operation; furthermore, it slows the rate at which the grids shrink between layers. Commonly, CNN employs a pooling layer after each convolutional layer to compress the grids. However, this is not necessary, and the paper [15] does not use this pooling layer at all, relying solely on the stride to reduce the size of the feature maps. After performing 3 convolutional layers, with ReLU activation functions, paper [15] feeds their extracted features to their fully connected layer (that can be viewed as an MLP), which maps these features to Q-values for each action.

2.2.6 DQN Training Problems and Solutions

The authors of [15] state that nonlinear function approximation, such as a DQN, can cause instability or divergence in RL; therefore, they propose several methods to address this problem.

2.2.6.1 Experience Replay

A major issue the paper [15] mentions is that observation sequences are correlated, since in typical online RL, the agent learns from consecutive transitions, making consecutive samples influenced by and similar to one another. Additionally, since the policy is derived from the current Q-values, small changes to the Q-values can alter the policy and, consequently, the subsequent data distribution, creating negative feedback loops. To mitigate this, paper [15] introduces experience replay. Each episode transition $e_t = (s_t, a_t, r_t, s_{t+1})$ is stored in a large replay memory buffer. That is then used for Q-learning updates, where we randomly select minibatches from the pool of stored samples “*thereby removing correlations in the observation sequence and smoothing over changes in the data distribution*”[15].

2.2.6.2 A Separate Target Network

Another major issue paper [15] mentions is “*correlations between the action-values (Q) and the target values $r + \gamma \max_{a'} Q(s', a')$* ”. If the network is used both to estimate Q-values and to construct the target, updating the Q-value estimate can alter the target, potentially causing the network to oscillate or diverge. Therefore, the paper [15] proposes using a separate target network, which is updated to match the Q-network’s parameters after a fixed number of steps.

$$L_i(\theta_i) = \mathbb{E}_{(s,a,r,s') \sim U(D)} \left[\left(r + \gamma \max_{a'} Q(s', a'; \theta_i^-) - Q(s, a; \theta_i) \right)^2 \right] \quad (2.5)$$

Where at iteration i , $L_i(\theta_i)$ is the loss function minimized with respect to the Q-network parameters θ_i . The target Q-network is denoted with parameters θ_i^- . The expectation $\mathbb{E}_{(s,a,r,s') \sim U(D)}$ denotes that training transitions are “drawn uniformly at random from the pool of stored samples” [15].

2.2.6.3 Double DQN

Paper [17] proposes altering the target used in (Eq. (2.5)), where the target network is used to both select the next action and to evaluate its value, which paper [17] shows can lead to “substantial overestimation” of action values. Instead, they propose to let the training Q-network decide on the greedy next action, while the target network evaluates that action, resulting in the following equation:

$$L_i(\theta_i) = \mathbb{E}_{(s,a,r,s') \sim U(D)} \left[\left(r + \gamma Q(s', \operatorname{argmax}_{a'} (Q(s', a'; \theta_i)); \theta_i^-) - Q(s, a; \theta_i) \right)^2 \right] \quad (2.6)$$

2.2.7 Partially Observable Markov Decision Process

In a standard MDP, the agent observes the complete current state at each time step. In contrast, in a Partially Observable Markov Decision Process (POMDP), the true state is not observable [18]. Therefore, POMDP extends MDP and is defined by the tuple $\langle \mathcal{S}, \mathcal{A}, T, R, \Omega, O, \gamma \rangle$, where Ω is the set of observations, and $O(s', a, o)$ is the observation function that gives $P(o|s', a)$ i.e., the probability of observing $o \in \Omega$, after taking action $a \in \mathcal{A}$, leading to state $s' \in \mathcal{S}$. Since the true state is not observable in a POMDP, the policy cannot map the state to an action but instead uses a belief state $b(s)$, which gives a probability distribution over potential states $s \in \mathcal{S}$. Paper [18] states that the agent does not need to have access to the full observation history because its current belief state already captures all relevant past information, i.e., in our case, the KBU function and, respectively, the BBU model, is a sufficient statistic for the initial beliefs and previous observations, and can therefore serve as input describing the current state to the DQN.

2.2.8 Deep Recurrent Q-Learning for POMDPs

Paper [5] argues that the method of the DQN paper [15], where we stack a fixed number of past time steps into the input, is limited. Since a fixed number of past time steps is seldom sufficient to characterize initial beliefs and all previous observations, this limits the agent's ability to reach an optimal policy. For example, in games with a stationary policy, i.e., there are potentially infinite time steps, any finite-sized window may be insufficient. Furthermore, even for finite-horizon games, using a window equal to the horizon can be impractical; for example, in each time step, papers [5, 15] use an input of size (84×84) . If we imagine the finite number of steps is 500, the input will be of size $(84 \times 84 \times 500 = 3,528,000)$, which is likely too large even to be fed as input and will significantly hinder the DQN from finding an optimal policy. Therefore, paper [5] introduces (DRQN), which modifies the DQN structure by replacing the first fully connected layer (which appears after the CNNs feature extraction) with a specialized type of RNN called a LSTM, followed by a fully connected linear layer that takes the LSTMs outputs as inputs and provides Q-values for each action in the current state; this architecture is trained end-to-end [5]. However, in our problem, this network architecture is problematic because the belief-update model and the knowledge-to-action mapping are not separated. In fact, the LSTM belief-update model in this case likely learns representations that convey information beyond just predicting the true hidden state.

2.2.9 Recurrent Neural Network (RNN)

In contrast to a feedforward architecture like an MLP, neurons in a recurrent architecture can connect back to other neurons within the same layer or earlier layers, allowing RNNs to display memory-like behavior. Paper [19] states that a dynamical system can be defined as $h_t = f_\theta(h_{t-1}, o_t)$, where $f_\theta(\cdot)$ is a fixed transition function across time, that updates the hidden state h_t , based on its previous value and the observation o_t , thereby the hidden state h_t summarizes past inputs and states [19]. Paper [19] states that an RNN is a dynamical system with weights, in which the parameters are shared across all states and all lengths, which gives it generalization properties. At time step t , input $o_t \in \mathbb{R}^d$, output $y_t \in \mathbb{R}^q$, and the hidden state $h_t \in \mathbb{R}^p$, the weight matrices $W \in \mathbb{R}^{p \times p}$, $U \in \mathbb{R}^{p \times d}$, $V \in \mathbb{R}^{q \times p}$, and biases $b_i \in \mathbb{R}^p$, $b_y \in \mathbb{R}^q$, then an RNN is described by Eqs. (2.7), (2.8), and (2.9). In this case, the activation function is the element-wise $\tanh(\cdot)$ since $i_t \in \mathbb{R}^p$.

$$i_t = Wh_{t-1} + Uo_t + b_i \quad (2.7)$$

$$h_t = \tanh(i_t) \quad (2.8)$$

$$y_t = Vh_t + b_y \quad (2.9)$$

2.2.9.1 Backpropagation Through Time

The main method for training an **RNN** is **Backpropagation Through Time (BPTT)**, extending regular backpropagation by adding the chain rule through time. Given T previous steps the loss function becomes Eq.(2.10) [19], where $\mathcal{L}_1$ is the current time step, and $\mathcal{L}_t$ is the previous $(t - 1)$ time step. **BPTT** then updates the **RNN** parameters using gradient descent [19].

$$\mathcal{L} = \sum_{t=1}^T \mathcal{L}_t \quad (2.10)$$

2.2.9.2 Vanishing or Exploding gradients

The paper [19] notes that when training **RNNs** with **BPTT**, gradients are propagated across many time steps, which means that the number of terms in the product grows substantially; so if each derivative is either slightly smaller or larger than 1, the entire chain grows either exponentially small or large. Paper [19] states that gradient vanishing is more common for **RNNs** than gradient explosions, and a vanishing gradient makes it difficult for the **RNN** to learn long-term dependencies.

2.2.10 Long Short Term Memory (**LSTM**)

LSTM offers a solution to the standard **RNN** problem with long-term dependencies; according to paper [19], **LSTM** “*learns from its input sequence when to use short-term dependency (i.e., when to clear the state) and when to use the long-term memory (i.e., when not to clear the state)*”. An **LSTM** cell contains an input gate described by Eq. (2.12), where $\odot$, is the Hadamard product and p_i is a learnable peephole weight, enabling a leak from the previous memory cell c_{t-1} , the input gate “*carries the importance of the information of the input at the current time slot*” [19]. Then we have the forget gate described by Eq. (2.13), which controls how much of the element-wise c_{t-1} should be carried over to c_t . Finally, we have the output gate described

by Eq. (2.14), which “considers the effect of the input and the previous hidden state, and a possible information leak from the current memory” [19].

$$\sigma(x) = \frac{1}{1 + \exp(-x)} \quad (2.11)$$

$$i_t = \sigma(W_i h_{t-1} + U_i o_t + (p_i \odot c_{t-1}) + b_i) \quad (2.12)$$

$$f_t = \sigma(W_f h_{t-1} + U_f o_t + (p_f \odot c_{t-1}) + b_f) \quad (2.13)$$

$$out_t = \sigma(W_{out} h_{t-1} + U_{out} o_t + (p_{out} \odot c_t) + b_{out}) \quad (2.14)$$

However, for the output gate, we first need to compute the new memory c_t , which first has another gate called the memory cell described by Eq. (2.15), and takes into account the “effect of the input and the previous hidden state” [19]. Now we can compute the new memory cell with Eq. (2.16), where the old memory depends on the forget gate, and the candidate memory depends on the input gate. We can then calculate the hidden state with Eq. 2.17, and the final **LSTM** output with Eq. (2.18), which can be identical to h_t by setting $V = I$ and $b_y = 0$.

$$\tilde{c}_t = \tanh(W_c h_{t-1} + U_c o_t + b_c) \quad (2.15)$$

$$c_t = (f_t \odot c_{t-1}) + (i_t \odot \tilde{c}_t) \quad (2.16)$$

$$h_t = out_t \odot \tanh(c_t) \quad (2.17)$$

$$y_t = V h_t + b_y \quad (2.18)$$

2.2.10.1 Modeling Missing Input Data with LSTMs

Paper [20] found that the best performing missing data strategy for **LSTMs** is to provide a missing measurement binary flag and to zero-fill the missing measurements, i.e., given an input variable $x_i^{(t)}$, the **LSTM** is given as input $[x_i^{(t)}, m_i^{(t)}]$, where $m_i^{(t)} \in \{0, 1\}$ if its missing or measured, and $x_i^{(t)}$ is set to zero if missing [20].

2.2.11 Related Work: Reinforcement Learning and Neural Nets

The following **RL** sections worked as an inspiration for investigating a different way of representing nested knowledge for the grid-based **PEG**, through an **LSTM-DQN** particle filter; however, it was not included in the final implementation, as the implemented particle filter did not show a significant performance gain and had to be abandoned due to time limitations. Therefore, readers uninterested in the additional details may skip the following sections on related work.

2.2.11.1 Reducing Model Overfitting

Paper [21] introduces label smoothing regularization (LSR) as a regularization technique that reduces model overfitting by replacing hard classification targets with softened target distributions. This method was originally used in the **BBU**-models but was later removed as no performance gain was observed. Given a training data point x , with labels $k \in K$, and a true normalized label probability $q(k|x)$, the neural network predicts the label probability $p(k|x) = \frac{\exp(z_k)}{\sum_i^K \exp(z_i)}$, where z_i are logits. With a cross entropy loss function $\ell = -\sum_{k=1}^K \log(p(k)q(k))$. Given a situation where the true probability $q(y) = 1$ and $q(k) = 0$ for all $k \neq y$ the log-likelihood is maximized for $q(k) = \delta_{k,y}$, where $\delta_{k,y}$ is the Dirac delta function e.g., (1 if $k = y$ else 0), can lead a model to become confident in wrong predictions. Therefore, paper [21] proposes LSR, which replaces the true label distribution $q(k) = \delta_{k,y}$ with Eq. (2.19), where $u(k)$ is a fixed distribution (during evaluation, they used a uniform distribution), and ϵ is a fixed weight.

$$q'(k|x) = (1 - \epsilon)\delta_{k,y} + \epsilon u(k) \quad (2.19)$$

Paper [21] notes that LSR discourages extremely peaked predictions since all $q'(k)$ have a positive lower bound, meaning the model is not encouraged to assign zero probabilities to all but one label, unlike $q(k)$, which equals 0 for all $k \neq y$.

2.2.11.2 Particle Filters

Paper [8] states that particle filtering is used for inferring the current state of an environment that changes over time using a sequence of measurements affected by noise. The goal of particle filtering is to approximate the posterior

$p(x_t|o_{0:t})$, since nonlinear or non-Gaussian models make exact analytical solutions difficult or impossible to obtain, where x_t is the hidden state, and $o_{0:t}$ are all of the observations made up to time step t [9]. Therefore, if x_t represents the teammates' inferred hidden state at time step t , then approximating the posterior $p(x_t|o_{0:t})$ would allow an agent to infer and update their second-order beliefs from observations over time. This idea is used in the paper [11], which introduces IPOMDP-Net, a neural network architecture that combines I-POMDP belief updates with QMDP planning. The model of [11] uses a particle-filter-style interactive belief update inside a RNN to approximate and update interactive beliefs, allowing an agent to maintain nested beliefs about other agents while being trained end-to-end. Meaning that the IPOMDP-Net architecture consists of both a memory update module and a knowledge-to-action mapping; however, because the architecture is trained end-to-end, the memory representation will be encouraged to encode information that is not human-interpretable.

Another approach taken by the paper [10] is to incorporate particle filtering into RNN to represent memory as a distribution over latent particles. Particle Filter Recurrent Neural Networks (PF-RNNs) replace standard RNN' deterministic latent state h_t , with a belief of h_t represented by K weighted particles $\{(h_t^i, w_t^i)\}_{i=1}^K$. Each particle is propagated through two learned transition functions. The transition update $\tilde{b}_t = f_{tr}(b_{t-1}, u_t(x_t))$, and the observation update $b_t = f_{obs}(\tilde{b}_t, o_t(x_t))$, where $u_t(x_t)$ is a learned latent control feature, and $o_t(x_t)$ is a learned latent observation feature, from the input data x_t . Therefore, if these particles represent the teammate agent, the transition function could be modeled by the agent's own first-order knowledge-to-action mapping. We could then split particles based on possible observations, as in the paper [11], and the resulting beliefs should represent second-order beliefs.

2.3 Explainable Artificial Intelligence

The goal of the thesis is to extract an interpretable strategy from a DQN black box; therefore, not all XAI methods apply to this problem, as some, such as feature importance, only show which factors are most important. Moreover, understanding how an AI reasons to arrive at its conclusions is not a trivial problem. Paper [7] states that for Deep Neural Network (DNN), it is relatively easy to understand why a node in the first hidden layer activates, and how activating a node in the last hidden layer affects the output. However, it is much harder to understand the intermediate layers, as they combine features in

intricate ways. Moreover, they state that the neural network contains nothing to distinguish meaningful patterns from spurious ones [7]. Going as far as to state “it seems wise to abandon the idea of trying to understand how a deep neural network makes a classification decision by understanding what the individual hidden units do” [7]. Therefore, explanations must operate at a different level of abstraction than the neurons do, such that humans can understand them.

2.3.1 Distilling a Neural Network Into a Soft Decision Tree

A classification **DT** is inherently interpretable, but requires labeled data because its training selects each branch split by measuring how well it reduces prediction error to the labeled dataset; without labels, e.g., not knowing before hand the optimal action for each state as in the case of trying to solve an **MDP**, there is no criterion to tell the **DT** which feature splits of a state will produce the optimal actions to achieve an optimal policy. Therefore, paper [7] proposes to create a soft **DT** trained with mini-batch gradient descent, where soft refers to probabilistic splits at the nodes, rather than deterministic splits (i.e., if a condition is met, we go right else left), trained to match the inputs to the **DNNs** respective outputs as the true label. This results in a **DT** that can model the **DNNs** predictions with an interpretable thought process. Moreover, even with a labeled data set, the authors of [7] show that a soft **DT** trained to mimic a **DNN** generalizes better than a **DT** trained on the true data set. However, they found that this **DT** solution performs slightly worse than the original black-box **DNN**.

2.3.2 INTERPRETER

Paper [6] introduces **INTERPRETER**, a method for extracting a strategy from a pretrained **DQN** oracle by training a **DT** classifier like in paper [7], but with deterministic instead of soft branching, and then converting the **DT** into Python code, allowing a human to edit the strategies. The authors of [6] evaluated their method on a few Atari games, such as Pong, and concluded that their solution matches the oracle’s performance. However, **INTERPRETER** is specifically designed for games formalized as **MDPs**. Consequently, the method does not address imperfect-information settings, such as our environment, where agents need to both maintain a knowledge state based on their entire sequence of observations and reason about the optimal action based on this knowledge state. Therefore, for **INTERPRETER** to apply

to our scenario, these two modules need to be separate, as it can only map knowledge to action; thus, it does not apply to an architecture such as a **DRQN**.

2.3.2.1 The Input Data

Paper [6] considers **MDP**-formalized Atari games, relying on predefined object-centric state representations provided by OCArari for both the oracle and the distilled policy. These symbolic features represent objects with their (x, y) coordinates, meaning that **INTERPRETER** relies on an externally specified symbolic representation that does not apply to our environment, as it has not been predefined for us. Paper [6] then simplifies the **MDP** state-action space by removing uninformative states (e.g., constant state features) and equivalent actions. Although the authors of [6] believe that symbolic representations are necessary for interpretability, they have limitations. Notably, if a relevant feature is absent from the representation, the distilled policy cannot use it. Furthermore, [6] argues that it is difficult to identify which features are critical to solve a task accurately, and that increasing the number of symbolic representations increases the complexity of the input space.

2.3.2.2 DAGGER

Paper [22] introduces an iterative imitation-learning method called **Dataset Aggregation (DAGGER)**; standard supervised imitation learners can suffer compounding errors, given a model that makes “a mistake with probability ϵ ”, after T time steps “can make as many as $T^2\epsilon$ mistakes in expectation” [22]. This compounding effect arises when a policy is trained only on expert demonstrations, since a deviation from the expert might cause it to visit states the expert rarely or never visits. The authors of [22] state that **DAGGER** “grows linearly in T and classification cost ϵ .” The **DAGGER** algorithm is quite simple; we start with an empty dataset $\mathcal{D}$ and an initial policy $\hat{\pi}_1$ (e.g., an untrained Neural Network). Then, for each iteration $i \in \{1, \dots, N\}$, we define a rollout policy π_i given in Eq. (2.20) that selects an action from the expert with probability β_i , and the learning policy otherwise [22]. The algorithm then samples T steps, for each step saving the state visited and the action the expert chose, then aggregating this sequence into the dataset $\mathcal{D}$, and training policy $\hat{\pi}_{i+1}$ on $\mathcal{D}$. The algorithm then returns the best $\hat{\pi}_i$ based on the validation data.

$$\pi_i = \beta_i \pi^* + (1 - \beta_i) \hat{\pi}_i \quad (2.20)$$

2.3.2.3 Q-DAGGER

Paper [23] introduces Q-DAGGER, a continuation of DAGGER, which uses the experts' Q-function to weigh imitation errors by the cost of picking the deviating action instead of the optimal one, expressed in Eq. (2.21). Paper [23] states that if we let Eq. (2.22) “be the 0-1 loss and $\tilde{g}(s, \pi)$ a convex upper bound (in the parameters of π), e.g., the hinge loss”[23]. Then if we let $\tilde{\ell}_t(s)\tilde{g}$ be defined by Eq. (2.23), the convex loss function becomes $\tilde{\ell}_t(s, \pi) = \tilde{\ell}(s)\tilde{g}(s, \pi)$, meaning that Q-DAGGER focuses learning on states where a wrong action would significantly impact performance. Moreover, Q-DAGGER differs from DAGGER by allowing the initial rollout policy $\hat{\pi}_1$ to be fully controlled by the oracle and does not sample actions from the oracle during further rollouts.

$$\ell_t(s, \pi) = V_t^{\pi^*}(s) - Q_t^{\pi^*}(s, \pi(s)) \quad (2.21)$$

$$g(s, \pi) = \mathbb{I}[\pi(s) \neq \pi^*(s)] \quad (2.22)$$

$$\tilde{\ell}_t(s) = V_t^{\pi^*}(s) - \min_{a \in A} Q_t^{\pi^*}(s, a) \quad (2.23)$$

2.3.2.4 INTERPRETER's Oracle Data

Paper [6] chooses between the imitation learning methods DAGGER and Q-DAGGER depending on the setting: Q-DAGGER is used when the Q-values are accessible, and the action space is discrete. Both of these aspects are fulfilled in our environment, since we have access to the Q-values and actions are in (up, down, left, right). When this is not the case, paper [6] uses DAGGER instead. According to the paper [23], Q-DAGGER has stronger performance guarantees than regular DAGGER; however, paper [6] states that from their experiments, they did not observe a difference in performance between the two, given equal prerequisites.

2.3.2.5 Oblique Decision Trees

Paper [6] argues that DQN policies often base their decisions to perform a certain action in a state on a combination of state features rather than one threshold over multiple individual features, e.g., $(My_x - Evader_x < 2)$ would be an oblique feature, if true denoting that the agent is at most 1 x-position away from the evader. Therefore, [6] argues that DTs, which only test for individual state features such as $(My_x < 2)$, are unlikely to capture these relationships between state features that the DQN learns and utilizes. Thus

[6] augments the Classification And Regression Tree (CART) algorithm to be able to return oblique **DTs**. For intuition, this amounts to increasing the feature space by pairwise comparisons of all possible features, with the drawback that the input feature space grows quadratically. The resulting oblique **DTs** consist of both single-feature conditions such as $(My_x < 2)$ and combinations such as $(My_x - Evader_x < 2)$. These oblique **DTs** can then be translated into a Python program by mapping the **DTs** internal nodes to if-else statements, and then pruned to merge redundant branches [6].

2.3.3 Related Work: Explainable Artificial Intelligence

The following related work sections are not directly relevant to the implemented solution but provide background on what is possible, and may therefore be skipped by readers who are not interested in the additional details.

2.3.3.1 Feature-Importance Explanations

Paper [2] illustrates why **XAI** is valuable through introducing two feature-importance explanation methods. The authors of [2] illustrates cases where a wolf-versus-husky neural network image classifier achieves seemingly strong performance by learning to rely on the typical snowy background characteristics of the images with wolves to determine if the animal is a wolf or a husky, rather than the animals' characteristics themselves, leading to poor generalization when snow is no longer a reliable cue, where the two feature-importance explanation methods can help humans understand why this failure occurs [2]. First, they introduce the method Local Interpretable Model-Agnostic Explanations (LIME), which explains the predictions of any classifier by fitting a simple, interpretable model locally around the input data. Secondly, they propose SubModular Pick LIME (SP-LIME), which selects a representative subset of instances and their explanations to provide a broader view of the model's behavior [2]. However, the drawback of this solution is that the model explains only the most important input features to conclude the outcome for a given data point. Therefore, these methods cannot extract an interpretable strategy from a **DQN**; if we were to use feature-importance explanations for our problem type, we could only highlight which probabilistic states are most important for determining the best action in any given belief state.

2.3.3.2 A Unified Approach to Interpreting Model Predictions

Paper [24] introduces SHapley Additive exPlanations (SHAP), a unified framework for interpreting model predictions, akin to feature importance explanations in [2], and is therefore not applicable to our problem. Paper [24] states that given a complex neural network, an alternative explanation model is needed because the original model's internal structure is not easily interpretable. Therefore, we must use an interpretable model $g(x')$ that can accurately approximate the original model $f(x)$ [24]. Where x' is a simplified version of x with a complete mapping function $x = h_x(x')$. The goal of a local explanation model is to ensure that $g(z') \approx f(h_x(z'))$ for inputs (z') that are close to the simplified input (x') [24]. Meaning that an explanation model $g(\cdot)$ only needs to approximate the original model $f(\cdot)$ locally, near the input data being explained. Therefore, SHAP produces an interpretable explanation model for a specific prediction rather than achieving our goal of extracting a full generalizable strategy.

2.3.3.3 SkillTree an Abstract DT

To combat the challenges of Long-horizon tasks, with high-dimensional state spaces and continuous action spaces that a DT is not well-suited for, paper [25] proposes a model called SkillTree. SkillTree introduces the concept of skills as “*temporal abstractions of low-level actions*”, meaning that the actual behavior remains hidden, and we instead understand which skills the neural network chooses between [25]. In SkillTree, these skills are represented as learned latent embeddings, which are not human-understandable. Therefore, although this potentially might lead to a better-performing DT than in [6, 7], it is not an explicit interpretable policy that a human can understand and follow, since what these skills represent, i.e., the learned latent embeddings, are not inherently interpretable, in contrast to the decision splits in the papers [6, 7].

2.3.3.4 Symbolic Representations

In order for a human to understand what a concept or a skill means, explanations must operate at a different level of abstraction than the neurons do, meaning symbolic representations are necessary for interpretability [6, 7]. Paper [12] proposes symbolic representations in which the environment is divided into fixed cells using K-means clustering to extract the belief space partition. The fixed belief-space partitions are defined as $\Phi = \{R_i\}_{i=1}^K$, and are calculated from K-means on weighted particles. In paper [12], $\Theta_K^{(i)} =$

$Pr(x_k \in R_i)$, denotes the probability mass $\Theta_k^{(i)}$ for cluster i at time step k . The paper then maps each cell probability $\Theta_k^{(i)}$ to boolean predicates by thresholding it into low-, medium-, or high-confidence levels. These belief predicates are then combined with depth predicates to form the symbolic state used when acting. This partition works well for paper [12] since, once an agent is inside a cell, its location within it does not matter. However, for us, this will not work, as the exact state matters; e.g., we cannot just assign a probability to a fixed (3×3) symbolic cell state in the top-left corner without also informing a pursuer exactly where the evader likely is within the fixed (3×3) . Moreover, if we assign every state a cell, then this symbolic representation becomes redundant.

Although the bachelor’s thesis [26] provides a symbolic representation of knowledge-based strategies for grid-based PEG, it is not particularly concise: it models an agent’s potential positions using two rhombi, with one describing the center position (initialized as unknown) and the other the potential spread from the center, meaning the worst case covers the entire grid, without any distance weighting.

2.3.3.5 Symbolic Predicate Invention

Paper [27] proposes a method called EXPIL to learn interpretable, logic-based policies through predicate invention from a pretrained neural policy such as an oracle DQN. The method involves extracting object-based concepts (i.e., distance, direction, existence) from an oracle network to create predicates and rules [27]. However, it seems inefficient; from their experiments, some results were lackluster. Furthermore, we need to define the objects first. Moreover, the maximum number of input objects in their experiments was only 5. Furthermore, in one experiment with only 4 total objects (where objects, for example, represent information about an agent), EXPIL generates more than 800 predicates to measure the agent’s distance and direction relative to the other 3 objects [27].

2.3.3.6 Extracting a Mathematical Explanation

Paper [28] introduces a symbolic metamodeling framework that can convert a black-box into a white-box mathematical function. The authors of [28], illustrate how they turn an MLP depicted as $f(x_1, x_2)$, with 2 hidden layers taking two inputs into an approximate mathematical solution $g(x_1, x_2) = x_1 x_2 (1 - x_2 e^{-x_1})$. Paper [28] introduces $G(x; \theta)$ as a Meijer G-function-based parameterization that allows extracting a mathematical function

approximation of the oracle network, such as a polynomial, exponential, logarithm, and more. However, the extracted functions produce a single continuous value for a given input, making it unclear how to adapt this method to our environment, where the **DQN** chooses among multiple actions in a given state. Moreover, since [28] did not test their implementation in an **RL** setting, and their neural network architecture was quite small, the potential performance in our environment is unclear.

2.3.3.7 Learning Finite State Representations of Recurrent Policy Networks

In a recurrent policy network such as **DRQN**, in each time step, the **RNN** receives an observation, extracts the observation features, and, based on the hidden state, outputs an action ($a_t = \pi(h_t)$). Therefore, paper [29] states that creating an interpretable policy for an **RNN** is difficult due to this dependency on continuous-valued memory representations and observation features. To address this problem, the authors of [29] introduce an approach to transform **RNN**-based policies with continuous memory and observations into finite-state Moore Machine representations, using a Quantized Bottleneck Network (QBN). A QBN is an Autoencoder (AE), where the latent layer (the bottleneck) is composed of k -level activations; the paper used 3-level quantization $\{-1, 0, +1\}$, allowing the QBN to convert continuous values into discrete representations [29].

$$b(x) = D(\text{quantize}(E(x))) \quad (2.24)$$

$$\phi(x) = 1.5\text{tanh}(x) + 0.5\text{tanh}(-3x) \quad (2.25)$$

Where $b(x)$ described in Eq. (2.24) is the output of the QBN, and D and E are continuous multilayer decoder and encoder, respectively. The encoder in the paper uses Eq. (2.24) as the activation function to output continuous values in $[-1, +1]$; they do not explicitly state how *quantize* transforms the values to $\{-1, 0, +1\}$, but it is likely just the nearest integer. Paper [29] states that quantization makes the network non-differentiable; i.e., the gradients become 0 between the encoder and decoder. To get around this, paper [29] uses “*the standard straight-through estimator*”, which treats the quantization function as the identity function during backpropagation. The QBN can then be trained as an AE with the L_2 reconstruction error $\|x - b(x)\|^2$ [29]. Paper [29], then proposes that two QBNs one for the feature vector f_t , and one for the hidden state vector h_t , can be inserted into the **RNN**, and given that the reconstruction

loss is zero, the policy of the network will be unchanged but, now its internal representations are quantized and can therefore be extracted as a finite state model. They call this a Moore Machine Network (MMN). However, although the extracted MNM provides a finite-state representation of the **RNN** policy, the meaning of the resulting states and observation symbols is not explicit; the authors state that “*solving the full interpretation problem in a primitive way is beyond the scope of this work*”. Therefore, the method primarily helps identify whether a policy relies on memory, observations, or both; however, in our game we already know that policies rely on both memory and observations, and we want to extract an interpretable policy from the **DQN**. Therefore, this paper does not apply to achieving our goals.

Chapter 3

Methodology

This chapter provides an overview of the thesis’s contributions and the research methods used to evaluate the results. The full thesis implementation is available at [GitHub](#). For detailed information on the implementation’s design, see Chapter 4: “A Framework for Explainable Knowledge-Based Strategies”.

3.1 Overview of the Contributions

This thesis investigated how the state-of-the-art **XAI** methods can be combined with knowledge-based strategies on grid-based **PEG** to produce human-interpretable strategies. To discover which knowledge states cause agents to perform certain actions, such that a human can reproduce them. Furthermore, because previous work, such as [3], has not found convincing evidence that moving from a first- to a second-order knowledge-based strategy yields a substantial performance improvement. Raises the question of whether second-order agents actually utilize their second-order knowledge to make decisions. Consequently, the goals of the thesis include, first, (i) investigating whether we can achieve a statistically significant improvement when transitioning from a first- to a second-order knowledge-based strategy. Moreover, secondly, (ii), to develop a framework for extracting interpretable, knowledge-based strategies from **DQN**s to determine whether an agent with second-order knowledge uses it in its reasoning process, and make observations about how the agents use the knowledge. However, a formal analysis of how the agents use first- and second-order knowledge is beyond the scope of this thesis.

Accordingly, this thesis developed its own Python tool to simulate, train, evaluate, and distill interpretable knowledge-based strategies from **DQN**s on a (15×15) grid-based **PEG** with two pursuers and one evader. To enable the

distillation of second-order human-understandable strategies from the **DQN**, this thesis replaces the true set-based knowledge-based update (**KBU**) function used in [3] with an **LSTM**-based architecture because the **KBU** function scales poorly with nested beliefs, producing second-order knowledge representations whose size grows quadratically with the number of possible positions. For clarity, this thesis distinguishes between the true set-based **KBU** function and the **LSTM**-based architecture, which we refer to as the belief-based update (**BBU**) model. The first-order **BBU** model consists of a single **LSTM** that generates two probability maps over the possible positions of the evader and the teammate on the grid. While the second-order **BBU** model consists of two **LSTM**, i.e., it has an additional **LSTM** that approximates second-order knowledge (i.e., nested beliefs). The second **LSTM** takes input from the main agent's observation and the first **LSTM** and outputs two additional probability maps representing beliefs about beliefs. The thesis then introduces a new symbolic representation for knowledge-based **PEGs** that compresses the probability grids into weighted probability masses for each direction, sufficiently compressing the feature input space to enable the implementation of the **INTERPRETER** method, which outputs strategies as Python **DTs**. See Chapter 4 for design rationale and model details.

3.2 Benchmark Strategies

This thesis implements two benchmark strategies to serve as reference points for evaluating and validating the implementation results.

3.2.1 The First-Order **KBU-DQN** Strategy

Since this thesis replaces the old **KBU** function with an **LSTM** approximation (the **BBU** model), we need to ensure that the new model is not inferior to the old, as this would undermine the significance of our results. For this reason, the thesis also implemented an adaptation of the first-order **KBU** function, used in [3]. Moreover, in the spirit of interpretability, this thesis also plotted simulated games and their respective knowledge grids and identified a likely mistake in the definition of [3]'s **KBU** function. In [3], uncertainty for visible positions is only cleared after performing uncertainty propagation, which allows uncertainty in some cases to propagate into positions that are impossible. The simple solution is to clear uncertainty for visible positions both before and after uncertainty propagation. Fixing this mistake led to a significant performance improvement for the first-order **KBU-DQN** strategies.

3.2.2 The Naive Perfect-Information Strategy

The naive perfect-information strategy is simple: it always performs the action that minimizes the Manhattan distance to the evader, with a preference for minimizing the larger of the x- or y-distances. Although not an optimal strategy, since pursuers P1 and P2 do not consider each other when acting, e.g., they will not purposely encircle evader E1 to limit its possible escape routes. However, it still gives a sense of what the attractive metrics for capture rate and average steps-to-terminal-state are for a knowledge-based strategy, since the perfect information about E1's location gives the pursuers access to information beyond what a knowledge-based strategy can deduce. Moreover, because it is a naive distance-minimization algorithm, its performance will not be impacted by a poorly designed **DQN**.

3.3 Data Collection and Sample Size

Since the data is collected from simulations, the only limitation for the sample size is time constraints. Because we generate the samples randomly, there is no guarantee that the evaluation data will cover every possible sequence of game states. Therefore, this thesis evaluated a relatively large final sample size of 1,000,000 simulations for each model. Where all models are tested on identical unseen seeds, such that all models are tested on the same initial conditions, and for each game, we save the seed, whether the pursuers succeed or failed to capture the evader, and the combined number of steps the pursuers took to reach a terminal state, i.e., either the number of steps to capture the evader or reaching the maximum of a combined 100 pursuer steps.

3.4 Statistical Significance of the Results

To determine the statistical significance of capture rates, this thesis performs pairwise exact binomial McNemar tests for each model, since the data are categorical (capture or failure). To determine the statistical significance of the average number of pursuer steps to a terminal state, this thesis performs a paired t-test for each model. The functions are imported from SciPy, and in each case, we use the same seeds. Where the null hypothesis states that the models' performances are the same, and the alternative hypothesis states that they differ.

3.5 Model Selection of Decision Trees

This thesis investigates the performance of **DT**s of different sizes, as smaller **DT**s are inherently more interpretable. In contrast, a larger **DT** might better capture the oracle **DQN**'s behavior. The goal is for the distilled **DT** to maximize the capture rate and minimize the average number of pursuer steps to a terminal state. Following the Q-**DAGGER** algorithm [23], this thesis created 10 candidate **DT** for each number of leaf nodes in {128, 256, 512, 1024}, and picked out the best-performing **DT**, based on maximizing the same reward structure as the oracle **DQNs**. The best-performing candidate **DT** for each size and each oracle **DQN** model will be plotted, with comparisons of action agreement, capture rate, and average steps to a terminal state based on 40,000 evaluation simulations. Based on this plot, the most interesting **DT** size, which will be the same across the models, will be selected for statistical analysis on the same 1,000,000 held-out set of seeds as the oracle **DQNs**. However, this thesis will not perform a statistical significance test of action agreement on the models, due to time limitations, and because, firstly, some actions in some states have equivalent outcomes; secondly, performance is more important than action agreement; and finally, deviating from the oracle policy can lead to better performances in some cases.

Chapter 4

A Framework for Explainable Knowledge-Based Strategies

In the following sections, we outline the key design decisions of this thesis. This thesis implemented its own Python tool to simulate, train, evaluate, and distill interpretable knowledge-based strategies for grid-based pursuit-evasion games; the full thesis implementation is available at [GitHub](#).

4.1 Architectural Overview

This thesis presents a five-stage model-training pipeline for obtaining an interpretable distilled **DT** with second-order beliefs. The following sections will motivate each stage individually, while this section provides a brief overview to give intuition for how they fit together. In stage (i), we train the first-order **BBU** models (one **LSTM** for each pursuer) to classify the evader and teammates' likely positions on the grid, based on the acting agents' observation history. In stage (i), no agent utilizes knowledge, and all agents select actions randomly. In stage (ii), we train the pursuers' first-order belief-to-action models (their **DQN**s) taking belief inputs from their pretrained first-order **BBU** models. In stage (iii), we train the pursuers' second-order **LSTM** models; the pursuers act based on their first-order **BBU-DQN** pair; the second-order **LSTM** model tries to learn how to infer the teammates' first-order beliefs, based on the acting agent's observation history, and its first-order **LSTM**. In stage (iv), we train the pursuers' second-order **DQN** using belief inputs from their second-order **BBU** model, which comprises their pretrained first- and second-order **LSTM**. Finally, in the last stage (v), we distill interpretable belief-to-action **DT**s from the second-order **DQN**.

4.2 The PEG Environment

The game is played on a 2D grid with no internal walls of size (15×15) , i.e., there are 225 possible positions (depicted in Figure 4.1). The agents start at random positions on this grid, consisting of two pursuers, P1 and P2 (utilizing knowledge), who work together to capture the evader E1 (with no knowledge, moving at random, acting as nature) within 50 turns; otherwise, the evader wins. Each turn consists of P1 moving one step (e.g., up, down, left, right) within the borders of the 2D matrix, then P2 moves, and finally E1 moves. The agents have corridor vision, meaning they only observe positions on the same row or column as themselves. Since there are no walls, nothing can obstruct an agent's view along the same row or column. Furthermore, if another agent were within the observer's line of sight and then moved out of it, the observer would also observe the agent's new position (since they would have observed the agent's direction of movement). Likewise, if another agent enters the observer's field of view, the observer also learns the agent's old position. This provides a second-order strategy, with some information to reason about which underlying beliefs could have motivated the agent's action.

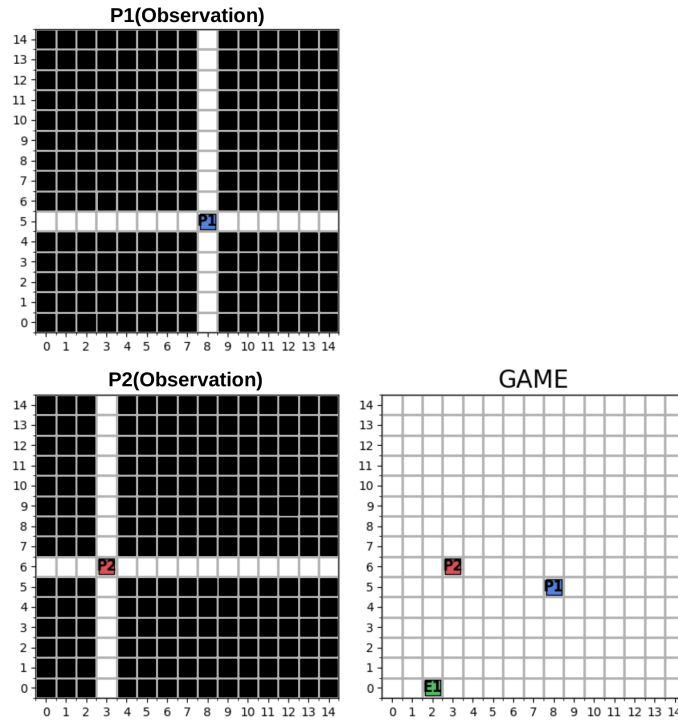


Figure 4.1: Depiction of the pursuer's current observation in the game; note that neither agent has observed the other entering or exiting its field of view.

4.3 Notes on Agent Independence

To preserve agent independence, each agent has its own belief- or knowledge-based update model and belief- or knowledge-to-action model. This thesis will follow the assumption of [18], **Model Non-Manipulability (MNM)**: an agent's actions cannot directly change another agent's internal model or beliefs, only indirectly through the environment. However, the assumption of **Model Non-Observability (MNO)**: an agent cannot directly observe another agent's internal model or belief [18], is not strictly followed during the training phase of the second-order **BBU** model, where the target probability distribution for the loss functions is the other agents' first-order beliefs, although training and evaluation of the second-order **DQN** remains non-observational because the agent no longer has access to the other agents internal models directly. We relax the **MNO** assumption during the training of the second-order **BBU** for two reasons: **(i)** the first-order beliefs are supposed to be an approximation of the publicly available true first-order **KBU** function. They should therefore not be specific to individual agents. Furthermore, **(ii)**, agents trained with **DQNs** are implicitly encouraged to learn other agents' behavior patterns, as this is beneficial for optimizing capture in a shared environment; therefore, this thesis assumes that the agents' strategies are public knowledge as well.

Thus, if we wanted to strictly follow **MNO** during the training of the second-order **BBU** model, we could either compute what the main agent would believe if they were in their teammate's position, but still let the teammate control the agent; this however will increase computation time, as now we would need to perform **LSTM** forward passes for an additional model, where time constraints are already an issue. Because of reason **(i)**, we know that the first-order **BBU** should not be agent-specific, so relaxing **MNO** here should not affect the result. Alternatively, both agents can be controlled by either P1's or P2's first-order **BBU-DQN** pair. However, the drawback of letting the main agent's strategy control both agents is that asymmetry in turn order can lead to differences in the agents' strategies; therefore, the second-order **BBU** might infer incorrect beliefs about the other agent's beliefs based on observing the other agent's actions. That said, based on limited experiments with both training configurations, no significant difference in performance was observed. Because of reasons **(i)** and **(ii)**, instead of potentially degrading the second-order **BBU** model in terms of performance or execution time, I will assume that the first-order **BBU** model that should approximate the first-order **KBU** function is also public information during the training phase of the second-order **BBU** model.

4.4 Knowledge-Based Update Function

The old first-order **KBU** function was implemented to serve as a baseline for testing against the first-order **BBU** model. It is a slight modification from [3], where each agent's possible location is represented by a separate probability map, instead of having one overlapping map where each state is symbolized by a different integer (e.g., an integer for unknown, possible evader state, possible teammate state, and wall). The true set-based **KBU** function works as follows: for the observing agent, for each probability map of the other agents, at the start of the game we initiate every state as possible, and set position uncertainty for all observable positions to zero, then each possible location is propagated to its valid move states (e.g., up, down, left, right), and the previous state is removed given that no other possible state-action pair could move the agent into that previous state. All observable positions are ruled out again as possible locations, since if an agent were there, the observer would see them. And since this update does not account for the probabilities of each state, the probabilities across possible locations are distributed uniformly and sum to 1. An example of the resulting knowledge states is depicted in Figure (4.2).

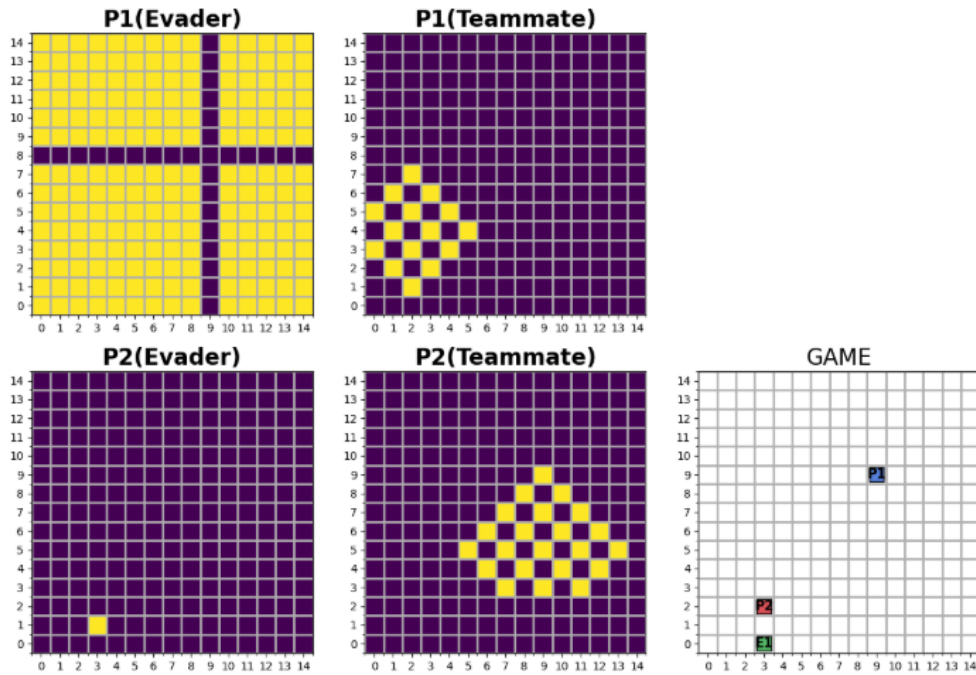


Figure 4.2: **First-Order Knowledge**: Note that an agent's depicted knowledge is saved before acting, so when another agent has acted afterward, the agent's depicted knowledge state will not be based on the current game state.

However, even if paper [3]’s second-order **KBU** function is correct, it scales poorly for nested beliefs, producing second-order knowledge representations whose size grows quadratically with the number of possible positions. For example, a (15×15) matrix with no walls has 225 possible positions; then, the knowledge update function of [3] produces an output of size 50,852 ($225 + 225^2 + 2$). The authors of [3] subsequently used this high-dimensional knowledge representation as input to their **DQN**. But this is problematic for us for three reasons: (i) in general, high-dimensional input data for a neural network requires more training. Furthermore, (ii), because a significant amount of the second-order inputs provides little information, the **DQN** can learn to ignore this data altogether. It is therefore possible that these two problems are the reasons [3] could not find a second-order strategy that shows a significant improvement over its corresponding first-order strategy. Moreover, (iii), most state-of-the-art policy-extraction **XAI** methods are highly limited in terms of how large the input space can be. Thus, I argue that the set-based knowledge representation of [3] is unsuitable for a second-order **XAI** strategy, as it results in a far too large second-order input space.

4.5 Belief-Based Update Model

This thesis approximates the specific grid-based **PEG KBU** function using **LSTMs**, given the problems with the second-order **KBU** function outlined in Section 4.4. Moreover, **LSTMs** were specifically chosen because **LSTMs** “*have been extraordinarily successful for prediction with sequential data*” [10], and we want to approximate knowledge from sequential observations.

4.6 First-Order Belief-Based Update

Following the architectural design of [5], the **BBU** model first encodes each observation into a hidden state vector. However, rather than using a **CNN** as in [5], the **BBU** model uses a single linear-layer feature encoder with a **ReLU** activation function. This hidden state vector is then passed to an **LSTMCell**, which maintains memory over the simulation sequence, producing another hidden state vector that is then fed into two separate linear transformations: one to predict logits over possible positions the teammate pursuer might occupy, and another to predict logits for the evader. These logits are then converted to probabilities using the PyTorch softmax function. The first-order **BBU** model architecture is trained end-to-end from observation to position logits.

Importantly, the **BBU-DQN** architecture is not trained end-to-end, as this allows us to abstract away the **BBU** model from the belief-to-action mapping we aim to explain, as paper [12] states our **DQN** can then take “*the expectation of the belief state as the true state, converting the problem into a deterministic search*”. If the **BBU-DQN** were connected end-to-end, the belief grids would likely be trained to encode information beyond state probabilities, which would then require explaining what they actually represent, which would be outside the scope of this thesis. Furthermore, we want these two networks to be trained for separate tasks: the **BBU** should be optimized only to produce an accurate, interpretable belief distribution; an example of a resulting first-order **BBU** output is depicted in Figure (4.3). In contrast, the **DQN** should map these beliefs to actions, such that we can extract an interpretable strategy from them.

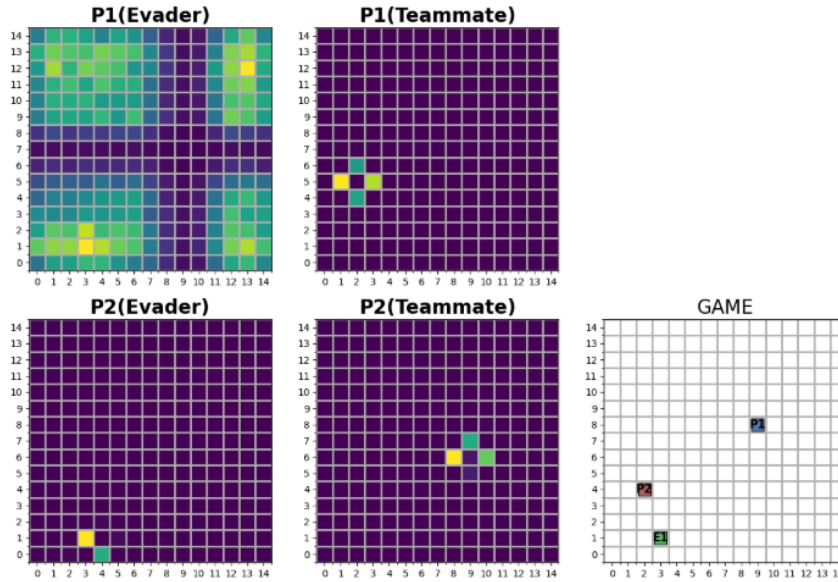


Figure 4.3: **First-Order Beliefs**: Note that an agent’s depicted beliefs are saved before acting, so when another agent has acted afterward, the agent’s depicted belief state will not be based on the current game state.

Crucially, all agents move randomly during training of the first-order **BBU** model. Most importantly, having the agents move randomly prevents the **LSTM** from learning and relying on any specific agent strategies, forcing it to learn only the agents’ movement dynamics, which the first-order **KBU** function also models. Moreover, random agent movement avoids a shifting-goalpost problem in state-probability assignment, since if the first-order **BBU** model were trained to learn the agent’s movement dynamics at the same time as the **DQN** is trained, the target distribution would change as the **DQN** improves.

The first-order **BBU** is trained in mini-batches of size 250 episodes, using **BPTT** for an agent over the entire sequence of transitions in each episode. Where beliefs are the predictions of the other agent's positions in the current state based on the sequence of observations, with cross-entropy loss minimization to their true positions, utilizing the **Adaptive Moment Estimation (Adam)** optimizer; with all major functions imported from PyTorch. The training was conducted over 12.5 million simulated episodes, with a hidden state size of 256 and a learning rate of 10^{-3} . Due to time constraints, a limited grid search of alternatives showed that these hyperparameters yielded the best performance when training time was factored in.

4.7 From Knowledge/Belief to Action

To map knowledge/beliefs to actions, we train a **DQN**. Since **DQN**s were used in prior knowledge-based **PEG** work [3], and the literature supports their use for our problem type [5, 15, 17]. All **DQN**-synthesized strategies use the same **DQN** structure, except that the second-order **DQN**s additionally take second-order beliefs as inputs. The **DQN** first uses a **CNN** to extract features from the input data. The **CNN** takes 3 channels for first-order strategies and 5 channels for second-order strategies, where each channel is either a knowledge/belief grid or a grid showing the main agent's position. For example, the **DQN** for P1's first-order **BBU-DQN** takes as inputs P1's belief grids shown in Figure (4.3) and the game grid. However, only P1's position is displayed on the game grid, even if another agent is within its field of view, because beliefs should be a complete description of its past and present observations.

The **CNN** applies three convolutional layers with **ReLU** activation functions, increasing the number of feature channels from (3 or 5) to 16, then 32 in the first and second convolutional layers, and downsampling in the second and third convolutional layers using a stride of 2. The extracted **CNN** features are then concatenated with a remaining time scalar and passed to a two-layer fully connected layer, which, for clarity, can be viewed as an **MLP** that outputs Q-values for each of the four actions. Concatenating a remaining-time scalar should allow the **DQN** to learn time-dependent behavior, such as potentially exploring more early in the episode and acting more greedily near the end. The **CNN** structure was inspired by [15], which used a similar setup for Atari games. However, since the investigated game environment only considers relatively small (15×15) grids, a purely **MLP**-based **DQN** yielded results similar to, but slightly worse than, those of the **CNN** setup in my limited testing.

The **DQN**s are trained in mini-batches with 128 state transitions using the **Adam** optimizer. We minimize the temporal difference error between the predicted Q-values and the target network's Q-values using the Huber loss, sampled randomly from an experience replay buffer with a capacity of 250,000 transitions, simulated over 500,000 games with a learning rate of 10^{-4} , employing a target network with the Double **DQN** update, with a ($\gamma = 0.99$), which might have been a bit too large.

The **DQN** rewards are as follows: a capture bonus of +5, failure to capture is -8, and a step cost of -0.15. These specific rewards were chosen to encourage the pursuers to achieve a capture rate close to 100% while keeping the average step count under 20. These benchmarks were identified in the naive perfect-information strategy. If the main goal was to find a guaranteed-winning strategy, without regard to average speed, one would focus more on the capture-failure loss, as the **DQN** would then learn that the primary objective is to capture the evader within the time limit. I chose to balance speed and capture rate to find a belief-based strategy that could rival a naive, perfect-information strategy on both metrics in this environment. I chose not to use any shaping rewards, such as rewarding the pursuer for moving closer to the evader, since I didn't want to introduce a heuristic that could interfere with the **DQN**'s ability to find an optimal strategy.

4.8 Second-Order Belief-Based Update

The second-order **BBU** model does not alter the first-order beliefs; instead, it incorporates the pretrained, frozen first-order **BBU** model and uses its outputs as inputs to the second-order **LSTM**. Inspired by greedy layer-wise pretraining of Restricted Boltzmann Machines in Deep Belief Networks, however, we use layer-specific targets and therefore do not have an end-to-end fine-tuning stage when stacking the **LSTMs** to produce hierarchical/nested beliefs.

Moreover, the second-order **BBU** also takes the main agent's first-order **LSTM**'s hidden state and cell state vector, along with the observation including observed actions and time left, as input, so that the second-order **BBU** can reason about why the other agent acted in a certain fashion, given that their actions have been observed. There are first three single linear-layer feature-encoders with **ReLU** activation functions for (i) the first-order beliefs, (ii) the first-order hidden-states and cell-states, and (iii) observations, also containing observed actions and time left. These three embeddings are then concatenated and passed through an additional single linear-layer feature encoder with a **ReLU** activation to produce a combined input-candidate representation.

We then introduce a learned single linear-layer feature-encoder gate with sigmoid activation to regulate information flow between the first- and second-order **LSTM**s. Since several inputs to the second-order **LSTM** are derived from the same underlying observation history, such as the first-order hidden states, cell states, and the belief logits predicted from them, the combined input-candidate representation likely contains redundant information. The logic of the gate is therefore to allow the model to reduce noise before feeding it into the second-order **LSTM**, and, from limited testing, introducing this gate slightly improved the results.

During training of the second-order **LSTM**, the agents load their pretrained first-order **BBU-DQN** pair, and the second-order **LSTM** is then trained to minimize the Kullback-Leibler divergence between the second-order outputs and the other agents' first-order beliefs. The second-order **LSTM** has a hidden state size of 512 and is optimized with **Adam** with weight decay, using a small weight decay of 10^{-4} as a regularizer, and is trained for 2,000,000 games with a learning rate of 10^{-4} , which is smaller than that of the first-order **LSTM**. The hyperparameters were chosen based on a limited grid search. An example of the resulting belief states is depicted in Figure (4.4).

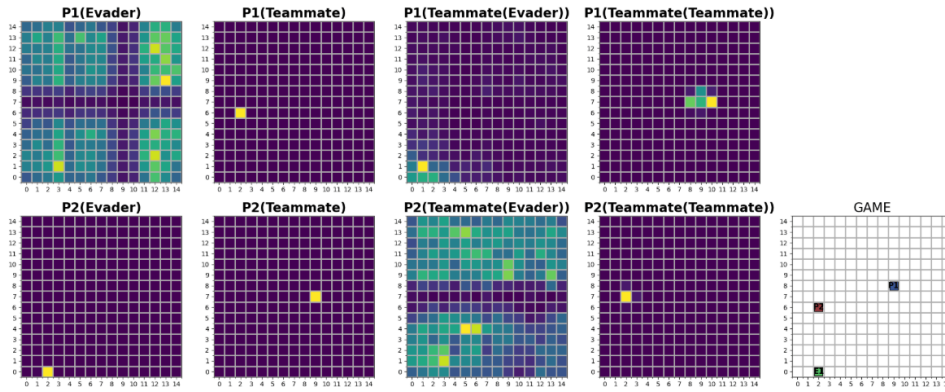


Figure 4.4: **Second-Order Beliefs**: Note that an agent's depicted beliefs are saved before acting, so when another agent has acted afterward, the agent's depicted belief state will not be based on the current game state.

4.8.1 Notes on Second-Order Particle-Filter Beliefs

Before deciding to use a stacked **LSTM** for hierarchical beliefs, this thesis first investigates the use of a particle filter. A major reason for investigating the use of particle filters was that the paper [10] achieved better belief approximations by incorporating a particle filter with their **LSTM** on highly variable data.

The implementation was inspired by IPOMDP-Net [11], in which we use hypothetical particles to represent possible teammate positions. Since the real observations of these particles are unknown, we split each particle’s propagation into those that would observe the evader and those that would not. However, I did not find a particularly strong method for generating fake observations, and this heuristic is limited because it becomes untenable to simulate every possible teammate observation at every possible location.

Possible evader positions and particles representing possible teammate positions would then be weighted based on the main agents’ first-order beliefs. Each particle would have its own pretrained first-order **BBU-DQN** pair, copied from the main agent, which generates the particle’s beliefs and actions. The particle filter would then compute a weighted mean belief from the particles, representing the main agent’s second-order beliefs. The idea was that, given enough particles and good resampling of the particles, their mean belief would converge towards the true teammate beliefs. However, the implementation was very slow, even if run in batch mode, because with 500 particles, we needed to perform 500 first-order **BBU-DQN** forward passes for each second-order belief update. Furthermore, the implemented second-order **PF-DQN** did not display a significant improvement over the first-order **BBU-DQN**. For these reasons and due to time constraints, the second-order particle filter approach was abandoned in favor of stacking **LSTMs**.

4.9 Extracting an Interpretable Strategy

To extract an interpretable **DQN** strategy, this thesis uses the **INTERPRETER** method introduced in [6] (see Section 2.3.2), which utilizes imitation learning to distill a deep **RL** policy into compact, editable **DT**-based policies in Python code, enabling inspection, modification, and testing of the policy in a human-understandable format. Paper [6] argues that symbolic representations are necessary for interpretability but have limitations. Notably, if a relevant feature is absent from the representation, the distilled policy cannot use it.

However, we cannot have a fully detailed symbolic representation for each individual state, like the **DQN**, since for our (15^2) grid environment, we take inputs of size $(1 + 15^2 \times 3)$ and $(1 + 15^2 \times 5)$ for the first- and second-order **DQNs**, respectively. Moreover, because **INTERPRETER** captures the concept that **DQN** policies select actions based on a combination of state features by creating oblique features, from pairwise combinations of the symbolic representation, given p features, the candidate feature space size is $\frac{p(p+1)}{2}$.

Consequently, having $p = (1 + 15^2 \times 5)$ becomes infeasible; however, information about every possible probability state is essential to the final interpretable **DT** strategy. Therefore, the problem becomes: how can we compress the input space to a manageable size for **INTERPRETER**, and still keep information about every state, given that [6] argues that it is difficult to identify which features are most critical for accurately solving a task.

4.10 A Symbolic Representation

This thesis addresses the symbolic representation problem by using scalars to represent time left and the agent's row and column positions, and by compressing each belief grid into a weighted-sum probability for each of the four directions (left, right, down, up), relative to the agent. All probability states in each knowledge/belief grid are first weighted by a Laplacian kernel $K(x, y) = e^{-\gamma \|x - y\|_1}$, with $\gamma = 0.1$. The Laplacian kernel gave better results from limited testing than, for example, weighting states by $\frac{1}{d+1}$, where $d = \|x - y\|_1$, i.e., the Manhattan distance. Then we divide the grid into all states that are in one of the four directions; this means, for example, that states in 2 directions, such as down and left, will be accounted for in both $P(\text{down})$ and $P(\text{left})$. Notably, this symbolic representation does not scale with the number of possible states; instead, it scales with the number of belief grids. Therefore, creating a symbolic representation of sizes 11 and 19 for the first- and second-order **DQN** to **DT** distillations, respectively. However, there is a risk of losing key features when performing this drastic feature compression.

4.11 INTERPRETER

Using this symbolic representation, the **Q-DAGGER** method was used to collect and weight state-action samples from the trained first- and second-order **DQNs** [23]. This thesis weighted state-action samples by their average advantage relative to the worst Q-value. The labeled state-action dataset built with **Q-DAGGER** was then used to train a **DT** that mimics the corresponding knowledge-to-action oracle **DQN** model; the implementation used `DecisionTreeClassifiers` from `sklearn`. The resulting **DT** strategies were then pruned to remove redundant branches and converted into Python code. A very small example of the resulting Python code is depicted in Figure (4.5). Upon inspecting the **DT** code, we can clearly observe that this strategy only considers the evaders' relative likelihood position.


```

7  def interpretable_strategy(features):
8      if features["Evader(LEFT) - Evader(RIGHT)"] <= 0.272517:
9          if features["Evader(UP)"] <= 0.044846:
10             if features["Evader(DOWN) - Evader(RIGHT)"] <= 0.017468:
11                 return 3 # WALK RIGHT
12             else:
13                 return 1 # WALK DOWN
14         else:
15             return 0 # WALK UP
16     else:
17         if features["Evader(DOWN) - Evader(LEFT)"] <= -0.067125:
18             return 2 # WALK LEFT
19         else:
20             return 1 # WALK DOWN
21
22
23 def interpretable_action(Evader_Probability_Grid, Teammate_Probability_Grid,
24                          Main_Agent_Position, Time_Left, Gamma, Size, Valid_Actions):
25     input_representation = symbolic_representation(Evader_Probability_Grid, Teammate_Probability_Grid,
26                                                  Main_Agent_Position, Time_Left, Gamma, Size)
27     input_combinations = get_feature_vector(input_representation)
28     symbol_to_value = {name: input_combinations[i] for i, name in enumerate(symbol_names)}
29     action = Index_to_Action[interpretable_strategy(symbol_to_value)]
30     if action in Valid_Actions:
31         return action
32     else:
33         return random.choice(Valid_Actions)

```

Figure 4.5: A very small interpretable first-order, **KBU-DT** strategy with a max leaf node count of 8, pruned down to 5. Full code available at: [GitHub](#).

Chapter 5

Results and Analysis

5.1 Performance Comparison of Oracle **DQNs**

Table 5.1 depicts the results in terms of average pursuer steps to a terminal state, win rate, total number of games lost, and achieved **DQN** reward across one million simulations using an identical held-out evaluation set of seeds. Note that since the naive perfect-information strategy is not a **DQN**-based model, its reward was computed manually using the same simple reward equation as the **DQNs**, allowing its benchmarks to be compared under the same criteria. In Table 5.1, we observe that the naive perfect-information strategy performs best in all metrics. Furthermore, among the trained models, the second-order **BBU-DQN** achieves the lowest average number of pursuer steps to a terminal state, and the set-based **KBU-DQN** achieves the highest win rate. Among the trained models, the first-order **BBU-DQN** achieved the lowest combined reward, being slightly worse than the first-order **KBU-DQN**. In contrast, the second-order **BBU-DQN** achieved the highest combined reward of the trained models, indicating that its second-order beliefs helped it learn the most effective policy under the defined reward function that balances the trade-off between speed and win rate.

Model	Avg. Combined Pursuer Steps	Win Rate	Total Failures to Capture	Combined Avg. Reward
First-Order KBU-DQN	19.608	99.985 %	146	7.055
First-Order BBU-DQN	19.538	99.939 %	610	7.053
Second-Order BBU-DQN	19.402	99.945 %	555	7.075
Naive-Perfect Information	15.742	100 %	0	7.639

Table 5.1: **DQN** results after 1 million validation simulations; all models were simulated on the same held-out-evaluation set of seeds

5.2 Distilled Decision Tree Metrics by Size

Figure 5.1 presents the best-performing DTs of varying sizes based on 40,000 validation simulations. However, note that these validation results are averages from two cases: one with a DT strategy controlling pursuer P1 and an oracle DQN controlling P2, or vice versa. Thus, they do not accurately reflect performance when DT strategies control both agents. This validation is therefore intended for model selection and shows the relationship between action agreements and DT size. Figure 5.1 shows an almost linear relationship between action agreement and increasing the DTs size from 128 to 1024 leaf nodes, although the same is not true for capture rate and pursuer steps. Note where the y-axis begins and ends; e.g., the worst capture rate for any model is only about two percentage points lower than the best. Based on the results presented in Figure 5.1, the thesis chose to evaluate DT of size 512, highlighted by red circles. These DTs demonstrated particularly interesting metrics across the models, achieving a 100% capture rate for the first-order KBU-DT, achieving action agreement with their respective oracles of about 86% to 87%, and a relatively low average number of steps to a terminal state.

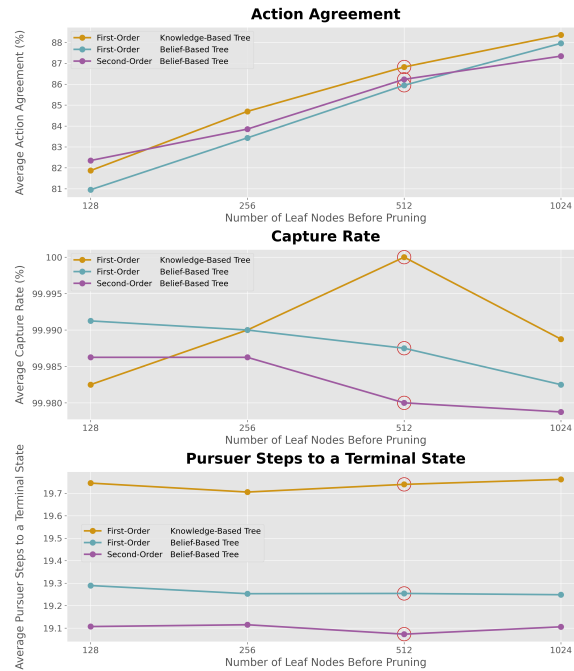


Figure 5.1: Relationships between DT size and (action agreement, capture rate, and pursuer steps to a terminal state) for distilled DT after 40,000 validation simulations. The red circle marks the selected DT for evaluation.

5.3 Performance Comparison of Distilled Decision Tree Strategies

Table 5.2 presents the results in terms of the average number of steps taken by pursuers to reach a terminal state, win rate, total number of games lost, and the achieved **DQN** reward across one million simulations, using the same seeds as the oracle **DQN**s held-out evaluation set. Note that none of the strategies employed here use the reward metric during training; instead, they are trained to mimic the oracle **DQN**s. However, during model selection for each **DT** size, the best-performing **DT** for that size (as shown in Figure 5.1) was chosen based on maximizing the reward metric. The reward was calculated manually using the same simple reward equation used for the **DQN**s, to allow comparison under the same criteria. Among the trained models, the second-order **BBU-DT** achieved the highest reward and the lowest average number of steps to a terminal state. Notably, both **BBU-DT** models improved across all metrics compared to their oracle counterparts. While the set-based **KBU-DT** decreased in average reward and steps to a terminal state, it achieved an almost guaranteed winning strategy, only losing two out of one million games

Model	Avg. Total Pursuer Steps	Win Rate	Total Failures to Capture	Combined Avg. Reward
First-Order KBU-DT	20.031	99.999 %	2	6.995
First-Order BBU-DT	19.082	99.996 %	44	7.137
Second-Order BBU-DT	18.976	99.991 %	90	7.151
Naive-Perfect Information	15.742	100 %	0	7.639

Table 5.2: Distilled **DT** results after 1 million validation simulations; all models were simulated on the same held-out-evaluation set of seeds

5.4 Inspecting the Decision Trees

The following sections present handpicked code snippets from the distilled **DT** shown in Table 5.2 to illustrate parts of its decision-making process and to demonstrate the same unintuitive behavior observed with the time-left scalar. Each **DT** has 512 leaf nodes before pruning, resulting in code files of about 1400 lines, which are too large to include in the report. These snippets represent only the final conditions in their branches; consequently, we need to keep in mind that many other preceding conditions are excluded. The complete code files are available at [GitHub](#).

5.4.1 First-Order KBU-DT

In Figure 5.2, we observe on lines 107 to 109 how the acting agent utilizes its first-order knowledge to identify the best action. In the provided code snippet, the agent checks whether the likelihood that the teammate is below it exceeds the likelihood that the evader is to the right of the acting agent. Additionally, the agent assesses whether the evader is more likely to be positioned to the right of the acting agent than the teammate is. Moreover, we notice some unintuitive behavior on line 109, where the agent subtracts its belief that the evader’s position is below it from the time-left scalar.

```

106         else:
107             if features["Evader(RIGHT) - Teammate(DOWN)"] <= -0.273395:
108                 if features["Evader(RIGHT) - Teammate(RIGHT)"] <= 0.189748:
109                     if features["Evader(DOWN) - Time_Left"] <= 0.330626:
110                         return 1 # WALK DOWN
111                     else:
112                         return 2 # WALK LEFT
113                 else:
114                     return 3 # WALK RIGHT
115             else:
116                 return 0 # WALK UP
117         else:

```

Figure 5.2: A code snippet from the first-order **KBU-DT**.

5.4.2 First-Order BBU-DT

In Figure 5.3, we observe on lines 1049 to 1052 that the acting agent utilizes its first-order beliefs to determine the best action. In the code snippet, the agent checks whether the evader is more likely to be above the agent than below, whether the teammate is more likely to be above or to the left of the acting agent, and whether the teammate is more likely to be below the agent than the evader. Furthermore, we once again observe unintuitive behavior on line 1051, where the agent compares its belief that the teammate is below it with a time-left scalar.

```

1048         else:
1049             if features["Evader(UP) - Evader(DOWN)"] <= 0.330281:
1050                 if features["Teammate(UP) - Teammate(LEFT)"] <= -0.000028:
1051                     if features["Teammate(DOWN) - Time_Left"] <= -0.099792:
1052                         if features["Evader(DOWN) - Teammate(DOWN)"] <= 0.000026:
1053                             return 0 # WALK UP
1054                     else:
1055                         if features["Evader(RIGHT) - Agent_Row"] <= -7.763288:
1056                             return 3 # WALK RIGHT
1057                         else:
1058                             if features["Agent_Row - Time_left"] <= 5.670000:
1059                                 return 1 # WALK DOWN
1060                             else:
1061                                 return 3 # WALK RIGHT
1062                 else:

```

Figure 5.3: A code snippet from the first-order **BBU-DT**.

5.5.1 Statistical Analysis of Capture Rates

5.5.1.1 Old Model vs. New Models

Table 5.3 presents the P-values for comparisons between the old **KBU-DQN** model and the new models. We observe that all differences in capture rate between the old and the new models are statistically significant, with the **BBU-DQN**s being meaningfully worse than the old model. All distilled **DT** strategies are meaningfully better than the old model, and the distilled first-order **KBU-DT** strategy achieves the highest capture rate among them all.

First-Order KBU-DQN	New Models	Capture Failures	P-Value
-	First-Order BBU-DQN	+464	$p < 0.0005$
-	Second-Order BBU-DQN	+ 409	$p < 0.0005$
-	First-Order KBU-DT	-144	$p < 0.0005$
-	First-Order BBU-DT	-102	$p < 0.0005$
-	Second-Order BBU-DT	-56	$p < 0.0005$

Table 5.3: Pairwise binomial McNemar test for win rate comparing the old model against the new models.

5.5.1.2 All Models vs. The Naive-Perfect Information Strategy

Table 5.4 presents the P-values for all models relative to the perfect-information strategy. We observe that all differences are statistically meaningful, except that the **KBU-DT** strategy is not meaningfully worse than the naive perfect model. However, since the McNemar test only considers cases where models disagree, the sample size is small, with only two cases.

Naive-Perfect Information	All Models	Capture Failures	P-Value
-	First-Order KBU-DQN	+146	$p < 0.0005$
-	First-Order BBU-DQN	+610	$p < 0.0005$
-	Second-Order BBU-DQN	+510	$p < 0.0005$
-	First-Order KBU-DT	+2	$p = 0.5$
-	First-Order BBU-DT	+44	$p < 0.0005$
-	Second-Order BBU-DT	+90	$p < 0.0005$

Table 5.4: Pairwise binomial McNemar test for win rate comparing all models against the naive perfect information strategy.

5.5.1.3 Comparison of the New Models

Table 5.5 presents the P-values for pairwise comparisons among all new models. We observe that all capture rates are statistically significant except for the comparison between the new first- and second-order **BBU-DQN**s.

Model 1	Model 2	Capture Failures	P-Value
First-Order KBU-DT	First-Order BBU-DQN	+608	$p < 0.0005$
First-Order KBU-DT	Second-Order BBU-DQN	+553	$p < 0.0005$
First-Order KBU-DT	First-Order BBU-DT	+42	$p < 0.0005$
First-Order KBU-DT	Second-Order BBU-DT	+ 88	$p < 0.0005$
First-Order BBU-DT	First-Order BBU-DQN	+ 566	$p < 0.0005$
First-Order BBU-DT	Second-Order BBU-DQN	+511	$p < 0.0005$
First-Order BBU-DT	Second-Order BBU-DT	+46	$p < 0.0005$
Second-Order BBU-DT	First-Order BBU-DQN	+520	$p < 0.0005$
Second-Order BBU-DT	Second-Order BBU-DQN	+465	$p < 0.0005$
First-Order BBU-DQN	Second-Order BBU-DQN	+55	$p = 0.1127$

Table 5.5: Pairwise binomial McNemar test for win rate comparing all new models.

5.5.2 Statistical Analysis of Steps to Terminal State

5.5.2.1 Old Model vs. New Models

Table 5.6 presents the P-values comparing the old **KBU-DQN** model to the new models. We observe that all differences in the average number of steps to a terminal state between the new models and the old model are statistically significant. All new models, except the first-order **KBU-DT** strategy, are statistically better than the old model, with the best performance coming from the second-order **BBU-DT**.

First-Order KBU-DQN	New Models	Avg. Steps to Terminal	P-Value
-	First-Order BBU-DQN	-0.07	$p < 0.0005$
-	Second-Order BBU-DQN	-0.21	$p < 0.0005$
-	First-Order KBU-DT	+0.42	$p < 0.0005$
-	First-Order BBU-DT	-0.53	$p < 0.0005$
-	Second-Order BBU-DT	-0.63	$p < 0.0005$

Table 5.6: Pairwise t-test for average steps to a terminal state comparing the old model vs the new models.

5.5.2.2 All Models vs. The Naive-Perfect Information Strategy

Table 5.7 presents the P-values for all models compared with the perfect information strategy. We observe that all models perform meaningfully worse than the perfect-information strategy in terms of the average number of steps to a terminal state.

Naive-Perfect Information	All Models	Avg. Steps to Terminal	P-Value
-	First-Order KBU-DQN	+3.87	$p < 0.0005$
-	First-Order BBU-DQN	+3.80	$p < 0.0005$
-	Second-Order BBU-DQN	+3.66	$p < 0.0005$
-	First-Order KBU-DT	+4.29	$p < 0.0005$
-	First-Order BBU-DT	+3.34	$p < 0.0005$
-	Second-Order BBU-DT	+3.23	$p < 0.0005$

Table 5.7: Pairwise t-test for average steps to a terminal state comparing all models against the naive perfect information strategy.

5.5.2.3 Comparison of the New Models

Table 5.8 presents the P-values for pairwise comparisons among all new models. We observe that all results on average steps to a terminal state are statistically significant, with the largest difference in average steps between the best performer, the second-order **BBU-DT** strategy, and the worst performer, the first-order **KBU-DT**.

Model 1	Model 2	Avg. Steps to Terminal	P-Value
First-Order KBU-DT	First-Order BBU-DQN	-0.49	$p < 0.0005$
First-Order KBU-DT	Second-Order BBU-DQN	-0.63	$p < 0.0005$
First-Order KBU-DT	First-Order BBU-DT	-0.93	$p < 0.0005$
First-Order KBU-DT	Second-Order BBU-DT	-1.06	$p < 0.0005$
First-Order BBU-DT	First-Order BBU-DQN	+0.46	$p < 0.0005$
First-Order BBU-DT	Second-Order BBU-DQN	+0.32	$p < 0.0005$
First-Order BBU-DT	Second-Order BBU-DT	-0.11	$p < 0.0005$
Second-Order BBU-DT	First-Order BBU-DQN	+0.56	$p < 0.0005$
Second-Order BBU-DT	Second-Order BBU-DQN	+0.43	$p < 0.0005$
First-Order BBU-DQN	Second-Order BBU-DQN	-0.14	$p < 0.0005$

Table 5.8: Pairwise t-test for average steps to a terminal state comparing all new models.

Chapter 6

Discussion

6.1 The Oracle DQNs

The performance comparison of the oracle DQNs showed that the models with second-order beliefs achieved the highest reward among all DQN variations, indicating that second-order beliefs give them an advantage. Moreover, the DQN statistical analysis showed that the second-order BBU-DQN showed statistically significant improvements in the average steps-to-terminal-state metric compared to other DQN models. However, in terms of win rate, it performed worse than the older first-order KBU-DQN and was neither statistically better nor worse than the first-order BBU-DQN. This result is likely due to the reward structure, where each agent gets (−0.15 step penalty, +5 for capture, and −8 penalty for failure to capture), since even though the second-order BBU-DQN secured the highest rewards and lowest average steps to a terminal state, it did not yield the best win rate, to test this, we ran the same experiment, only changing the reward structure to make the average steps to capture matter less and making failure to capture a catastrophic event (−0.01 step penalty, +100 for capture, and −1000 for failure). The results are shown in Table 6.1, and we observe that the second-order BBU-DQN achieves the highest reward and is the only DQN strategy to achieve a 100% win rate.

Model	Avg. Combined Pursuer Steps	Win Rate	Total Failures to Capture	Combined Avg. Reward
First-Order KBU-DQN	20.050	99.999 %	13	199.771
First-Order BBU-DQN	20.106	99.999 %	1	199.797
Second-Order BBU-DQN	19.702	100 %	0	199.803

Table 6.1: DQN results with different rewards, after 1,000,000 evaluation simulations, on the same held-out set of seeds.

6.2 The Second-Order Architecture

The stacked **LSTM** architecture is not fully optimized, so the second-order beliefs are less accurate than they could be. This thesis first devoted significant effort to implementing second-order beliefs using a particle filter. When that implementation fell through, there was little time to investigate the state of the art in stacking **LSTMs**. Therefore, due to time constraints, the second-order **LSTM** is given all available information as input, including the main agents' first-order beliefs and the **LSTM**'s hidden and cell states, resulting in many redundant, noisy inputs that stem from the same observation history. To address this problem, this thesis uses a learned gate between the **LSTMs** to filter out noise, though this implementation is likely not ideal. It did lead to a significant performance improvement, increasing the second-order agents' combined average reward by about 0.025 points, enough to move it from the worst to the best. A better-stacked **LSTM** architecture would likely yield even better second-order results.

Moreover, the first- and second-order strategies use the same trained model for their first-order beliefs (so their first-order beliefs are identical in all respects). All models have access to the same "observations," but with slight implementation differences for creating first and second-order beliefs; the difference is that the second-order **LSTM** has access to a time-left scalar and an explicit flag indicating teammates' observed actions, to help it reason about the other agents' state of mind. Adding the observed actions to the first-order **LSTMs** will not change anything, since the update is not intended to infer the other agents' states of mind that led to those actions. Furthermore, if an agent acts within another agent's field of view, the observing agent directly observes the other agent's new location, so there is no need for the observing agent to deduce the other agent's position from its actions. All first- and second-order models receive the remaining time scalar observation as input to their **DQN** (knowledge-to-action model), since agents might act differently based on how much time remains. It seemed possible that including a time left scalar in the input for determining the agent's state of mind could help; however, it is possible that this time-left scalar could give the second-order **LSTM** belief update an unfair advantage over the first-order **LSTM** belief update, since the first-order **LSTM** does not have access to the time left scalar, although an **LSTM** should, in principle, learn to track time elapsed internally, so this might not be a problem, moreover I did not test if including a time left scalar improved the second-order **BBU** model.

6.3 Distilled Decision Trees

Interestingly, simple **DT** strategies in the belief-based case outperformed their oracle **DQN** counterparts in all aspects. In contrast, the set-based **KBU-DT** reduced its average combined reward and increased the average number of steps to reach a terminal state, while achieving an almost-guaranteed winning strategy with a 99.999% win rate. It is possible that the set-based **KBU-DT** decreased in some aspects because the set-based **KBU** function is not well-suited to the probabilistic symbolic representation, as all possible states have equal probability of being true; this could cause the distance weighting to favor closer states more, leading to a greedier approach, even if these states' expected probabilities might be lower.

Using imitation learning such as the **INTERPRETER** method to distill a **DQN** policy into a **DT** with only 512 leaf nodes, likely acts as a form of regularization, therefore, the fact that the **DTs** performs better could suggest that the oracle **DQN** policies may have overfit to their training data, this idea of overfitting is also coherent with Figure (5.1), where we see action agreement increase with the number of leaf nodes, while the metric of average steps to capture and win rate, stay approximately the same or decrease slightly. However, with 512 leaf nodes, the decisions are hard to interpret. It might have been a mistake to make the **DT** this large, since Figure (5.1) did not indicate that the smaller **DT** in the range between 128 and 1024 performed significantly worse. In comparison, smaller **DT** are inherently more interpretable (such as the **KBU-DT** in Figure (4.5) with only 5 leaf nodes), with the drawback of a worse action agreement.

Another possibility is that the **DTs** benefit from a more compact symbolic representation that summarizes the entire possibility grid as four weighted probability masses for each possible direction. In contrast, the **DQNs** rely on **CNNs** to extract features from the complete possibility grid. Although this provides a more precise representation of the current state, it also introduces additional complexity, potentially making it harder for the architecture to learn true correlations and likely increasing the risk of overfitting.

Upon examining the resulting **DTs**, such as Figure 5.2, we observe that the first-order agents infer the game's state based on their beliefs about the positions of the other pursuer and the evader. For example, if the agent thinks the evader could be either to its right or below it, the agent examines the probability that the teammate is also below it or to its right to determine which direction is best. While the second-order strategy also reasons about the other agents' beliefs, such as in Figure 5.4, for example, the acting agent reasons

about whether the evader is likely above it, whether the teammate believes the acting agent is higher than it is, and whether the teammate is likely below the acting agent, when determining whether to walk up or down. The second-order beliefs seem to lead to better cooperation, since the second-order **DT** strategy achieved the highest reward and the lowest average number of steps to a terminal state among all imperfect-information strategies examined, at the cost of losing a few more games. Moreover, the second-order **DT** in Figure 5.4 appears to align with the given example in the definition of second-order knowledge.

Furthermore, upon inspecting the **DT**, we observe that some decisions are odd and likely suboptimal. Especially how **DTs** use the time-left scalar, it might have introduced noise, leading to spurious correlations. However, it is also possible that subtracting position possibilities from the time-left scalar is not intuitive but beneficial. Therefore, we tested a version that prevented **DT** from subtracting the time-left scalar and the agent’s row and column positions from any symbolic representation. We limited agent position comparisons also because positions are integers from 0 to 14, whereas position probabilities lie in $[0, 1]$. Table 6.2 presents the results of this experiment. We observed a slight performance improvement for the **BBU** models when limiting oblique features to probabilities alone, suggesting that the **DT** likely lacked sufficient data and therefore overfitted to its training data, leading it to make poor decisions and resulting in spurious correlations. This could suggest that, had we increased the amount of training data for the distilled **DTs**, we would have obtained statistically significant improvements in the **DTs**.

Model	Avg. Total Pursuer Steps	Win Rate	Total Failures to Capture	Combined Avg. Reward
First-Order KBU-DT	20.031	99.999 %	2	6.995
Limited -First-Order KBU-DT	20.050	99.999 %	13	6.992
First-Order BBU-DT	19.082	99.996 %	44	7.137
Limited -First-Order BBU-DT	19.014	99.997 %	28	7.147
Second-Order BBU-DT	18.976	99.991 %	90	7.151
Limited -Second-Order BBU-DT	18.960	99.997 %	28	7.155

Table 6.2: Limiting the **DTs**’ feature representation, disallowing it to subtract any symbolic feature except directional probabilities, i.e., the agent’s own position and the time left scalar, is only allowed to be compared standalone without subtraction.

6.4 Limitations

To obtain and evaluate a distilled **DT** with second-order beliefs, we first (i) need to train an **LSTM** for the first-order **BBU**, then (ii) train a **DQN** for the first-order belief-to-action mapping, then (iii) train another **LSTM** for the second-order **BBU**, then (iv) train another **DQN** for the second-order belief-to-action mapping, and finally (v) we can train and evaluate a distilled second-order **DT**. Depending on how rigorously we want to train the models to achieve optimal performance, this entire process for the thesis implementation can take up to a week. Meaning that, due to time constraints, many of the models are not fully optimized, as there was no time to perform grid searches to find the best hyperparameters for all models. Furthermore, the thesis selected the best **DT** for each **DT** size based on only 4000 validation simulations, which is likely insufficient to find the best **DT**. However, since we create 10 candidate **DT** for each size and test 4 **DT** sizes across 3 models, we evaluate 120 **DT**s, totaling 480,000 validation simulations for model selection, which is very time-consuming. Longer training and validation times would likely have yielded better **DT** strategies, possibly leading to more intuitive, knowledge- or belief-based decisions, but there was simply no time for this.

6.4.1 Ethics and Sustainability

From a consumption perspective, developing a simple **DT** strategy that outperforms the oracle **DQN** is good, as it is far more resource-efficient and requires far less compute; therefore, I think this thesis best touches on aspects of Sustainable Development Goal 12: Responsible Consumption and Production. However, creating these strategies in the first place can be quite resource-demanding due to their long training time, so it is a bit of a double-edged sword. From an ethical perspective, developing a human-interpretable strategy to track and capture someone who tries to get away can be morally gray, as such technology could be used on drones for warfare or terrorism. It could also be used in police work to help capture criminals. That said, the toy game examined here is unlikely to be ready for real-world deployment.

6.5 Reflections

Overall, I believe this thesis achieved its goal of developing a framework for obtaining interpretable knowledge-based strategies from **DQNs**. Due to numerous implementation steps and time constraints, many design decisions

were made hastily, leaving no time to thoroughly test all components for optimal behavior. My biggest regret is wasting so much time on the particle-filter idea, instead of looking into the state-of-the-art for stacking **LSTMs**, although when trying the particle-filter idea, many core design decisions were made, such as iteratively training the components (e.g., different orders of **BBUs** and **DQNs**), so that they can be abstracted away and used within the particle filter. Additionally, I think it might have been a mistake to train the first-order **BBU**-model with all agents moving at random; instead, it would have made more logical sense for the pursuers to use zero-order knowledge, i.e., performing actions based solely on the current observation. The original reason for not doing this was that the **KBU** function does not do it.

Chapter 7

Conclusions and Future Work

This thesis presented a framework for extracting interpretable knowledge- and belief-based strategies from **DQN**s for grid-based pursuit-evasion games. To enable the extraction of second-order strategies, this thesis replaced the old set-based **KBU** function with stacked **LSTMs**, creating separate belief grids for each modeled belief. This paper’s symbolic representation then compresses these probability grids into probability masses for each direction. The thesis showed that the extracted policies achieved an action agreement of around 86% and that the **BBU-DT**s outperformed the oracle **BBU-DQN**s across all metrics. We found an interpretable second-order strategy that reaches terminal states fastest on average while achieving the highest **DQN** reward, suggesting that second-order beliefs provide an advantage in achieving higher reward. Although the second-order strategies in general did not achieve the highest win rate, this is likely because the **DQN** reward favors reducing the average number of steps to a terminal state over achieving a 100% win rate during **DQN** training. Therefore, when altering the reward structure to prioritize win rate, we observed that the second-order **BBU-DQN** was the only **DQN** model to achieve a 100% win rate on the 1,000,000 hold-out evaluation seeds. Upon inspecting the resulting **DT**s, we found that the second-order strategies use their beliefs about others’ beliefs to make decisions. It seems that second-order beliefs convey information that enables agents to collaborate more effectively, potentially allowing them to communicate, to some extent, through body language. Interestingly, we also found a fully interpretable, from-observations-to-actions, first-order, knowledge-based strategy that was almost surely winning, losing only 2 out of 1,000,000 games.

Model	Avg. Total Pursuer Steps	Win Rate	Total Failures to Capture	Combined Avg. Reward
First-Order KBU-DQN	20.050	99.999 %	13	199.771
First-Order BBU-DQN	20.106	99.999 %	1	199.797
Second-Order BBU-DQN	19.702	100 %	0	199.803
First-Order KBU-DQN	19.608	99.985 %	146	7.055
First-Order BBU-DQN	19.538	99.939 %	610	7.053
Second-Order BBU-DQN	19.402	99.945 %	555	7.075
First-Order KBU-DT	20.031	99.999 %	2	6.995
First-Order BBU-DT	19.082	99.996 %	44	7.137
Second-Order BBU-DT	18.976	99.991 %	90	7.151
Naive-Perfect Information	15.742	100 %	0	7.639

Table 7.1: The most interesting results

7.1 Future work

In the following subsections, we present areas that remain unfinished or could yield more interesting results.

7.1.1 Improving Upon the Current Implementation

The complete implementation is not optimized; we did not have time, and there are several points for improvement. Such as tuning the rewards, improving the second-order **LSTM** loss function, and specifically optimizing the hierarchical **LSTM** architecture. The second-order **BBU** model could likely be substantially improved by future work that investigates state-of-the-art methods using stacked **LSTMs**. Furthermore, I believe that the way **INTERPRETER** is implemented in this thesis makes it vulnerable to selection bias, since it selects the best performing **DT**, while increasing the training data, on a relatively small validation test set, meaning a more general solution with more training data could, get outperformed by a more overfitted **DT**, that happened to be very good for the validation test set, of just 4000 simulations.

7.1.2 Analysing the **DTs**

Since the first-order **KBU-DT** only failed to capture 2 out of 1,000,000 simulations, and we also showed that the second-order **BBU-DQN** achieves a 100% win rate on 1,000,000 evaluation simulations if we tune the reward to make failure to capture a catastrophic event. I think it is highly likely we can find a 100% first-order **KBU-DT**, with just a few minor changes (e.g.,

tuning the reward slightly, and increasing the training data size of the **DTs**) and running a couple of more experiments. Therefore, future work could analyze such a guaranteed-winning strategy to produce proof that it always wins. Alternatively, future work could analyze the difference between the first- and second-order **BBU-DTs**, to investigate why the second-order **DT** achieves a slightly better performance, and if it is correct that the second-order **BBU-DT** agents achieve better cooperation.

7.1.3 XAI for the **BBU** model

One could implement **XAI** for the **LSTMs** in the **BBU** model. However, during the literature review, this thesis found no effective methods for doing so. Moreover, paper [29] states that **RNN** policies are “*particularly difficult to explain, understand, and analyze due to their use of continuous-valued memory vectors and observation features*”. The state of the art may not be there yet, so this could be a challenging or impossible task.

7.1.4 A Concise Second-Order **KBU**-function

Since the **BBU** models are not interpretable, if someone wants a fully interpretable function from observations to actions, such as the first-order **KBU-DT**, and does not want to try to make an additional **XAI** now instead for the **BBU** model, one could, for example, use a similar second-order version as [3]. Where [3] creates a belief-of-belief grid for each possible location an agent might occupy. However, before feeding the quadratic scaling representation with respect to possible positions into the **DQN** and the symbolic representation, we could compress every belief-of-belief grid into a mean belief grid. That way, we could create a fully interpretable second-order **KBU-DT**.

7.1.5 Higher Orders Strategies

This thesis’s method of stacking **LSTMs** and **DQNs**, iteratively first pretraining an **LSTM**, then pretraining a **DQN**, can be repeated to achieve higher-order strategies than just second-order; however, the training time required would increase, and it will also increase the number of symbolic features by 8 for each increased knowledge order, assuming we are only tracking one other pursuer’s beliefs.

7.1.6 Implementing the Particle-Filter Second-Order Belief Update

Although this paper failed to implement a high-performing second-order particle filter-based **BBU** model, other papers have indicated that this should be possible. Future work could implement this for knowledge-based **PEGs**.

7.1.7 Different Symbolic Representations

It makes sense to investigate different kinds of symbolic representations, or fine-tune the one presented in this thesis, to see if the **DT** achieves significantly better results. The thesis briefly tested a version that uses the same Laplacian kernel based on the agent's distance to the (left, right, bottom, top) walls of the matrix, so that the agent's position lies within the same domain as the probability masses. However, the thesis has not given this much thought, and based on limited testing, it did not show a substantial difference, for better or worse.

References

- [1] O. Arbman and O. Lilliestierna, “Effects of varied terrain on knowledge-based strategies in grid-based pursuit-evasion games,” 2024.
- [2] M. T. Ribeiro, S. Singh, and C. Guestrin, “‘why should i trust you?’: Explaining the predictions of any classifier,” in *Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, ser. KDD '16. New York, NY, USA: Association for Computing Machinery, 2016. doi: 10.1145/2939672.2939778. ISBN 9781450342322 p. 1135–1144. [Online]. Available: <https://doi.org/10.1145/2939672.2939778>
- [3] G. Forsell and S. Gergi, “Reinforcement learning for multi-agent strategy synthesis using higher-order knowledge,” 2023.
- [4] D. Gurov, V. Goranko, and E. Lundberg, “Knowledge-based strategies for multi-agent teams playing against nature,” *Artificial Intelligence*, vol. 309, p. 103728, 2022. doi: <https://doi.org/10.1016/j.artint.2022.103728>. [Online]. Available: <https://www.sciencedirect.com/science/article/pii/S0004370222000686>
- [5] M. J. Hausknecht and P. Stone, “Deep recurrent q-learning for partially observable mdps.” in *AAAI fall symposia*, vol. 45, 2015, p. 141.
- [6] H. Kohler, Q. Delfosse, R. Akrou, K. Kersting, and P. Preux, “Interpretable and editable programmatic tree policies for reinforcement learning,” *arXiv preprint arXiv:2405.14956*, 2024.
- [7] N. Frosst and G. Hinton, “Distilling a neural network into a soft decision tree,” *arXiv preprint arXiv:1711.09784*, 2017.
- [8] M. Arulampalam, S. Maskell, N. Gordon, and T. Clapp, “A tutorial on particle filters for online nonlinear/non-gaussian bayesian tracking,”

- IEEE Transactions on Signal Processing*, vol. 50, no. 2, pp. 174–188, 2002. doi: 10.1109/78.978374
- [9] O. Cappe, S. J. Godsill, and E. Moulines, “An overview of existing methods and recent advances in sequential monte carlo,” *Proceedings of the IEEE*, vol. 95, no. 5, pp. 899–924, 2007. doi: 10.1109/JPROC.2007.893250
- [10] X. Ma, P. Karkus, and D. Hsu, “Particle filter recurrent neural networks,” *Proceedings of the AAAI Conference on Artificial Intelligence*, vol. 34, pp. 5101–5108, 04 2020. doi: 10.1609/aaai.v34i04.5952
- [11] Y. Han and P. Gmytrasiewicz, “Ipomdp-net: a deep neural network for partially observable multi-agent planning using interactive pomdps,” in *Proceedings of the Thirty-Third AAAI Conference on Artificial Intelligence and Thirty-First Innovative Applications of Artificial Intelligence Conference and Ninth AAAI Symposium on Educational Advances in Artificial Intelligence*, ser. AAAI’19/IAAI’19/EAAI’19. AAAI Press, 2019. doi: 10.1609/aaai.v33i01.33016062. ISBN 978-1-57735-809-1. [Online]. Available: <https://doi.org/10.1609/aaai.v33i01.33016062>
- [12] M. Hou, T. X. Lin, H. Zhou, W. Zhang, C. R. Edwards, and F. Zhang, “Belief space partitioning for symbolic motion planning,” in *2021 IEEE International Conference on Robotics and Automation (ICRA)*, 2021. doi: 10.1109/ICRA48506.2021.9561121 pp. 8245–8251.
- [13] L. P. Kaelbling, M. L. Littman, and A. R. Cassandra, “Planning and acting in partially observable stochastic domains,” *Artificial Intelligence*, vol. 101, no. 1, pp. 99–134, 1998. doi: [https://doi.org/10.1016/S0004-3702\(98\)00023-X](https://doi.org/10.1016/S0004-3702(98)00023-X). [Online]. Available: <https://www.sciencedirect.com/science/article/pii/S000437029800023X>
- [14] C. Watkins and P. Dayan, “Technical note: Q-learning,” *Machine Learning*, vol. 8, pp. 279–292, 05 1992. doi: 10.1007/BF00992698
- [15] V. Mnih, K. Kavukcuoglu, D. Silver, A. A. Rusu, J. Veness, M. G. Bellemare, A. Graves, M. Riedmiller, A. K. Fidjeland, G. Ostrovski, S. Petersen, C. Beattie, A. Sadik, I. Antonoglou, H. King, D. Kumaran, D. Wierstra, S. Legg, and D. Hassabis, “Human-level control through deep reinforcement learning,” *Nature*, vol. 518, no. 7540, pp. 529–533, Feb. 2015. [Online]. Available: <http://dx.doi.org/10.1038/nature14236>

- [16] I. Goodfellow, Y. Bengio, and A. Courville, *Deep Learning*. MIT Press, 2016, <http://www.deeplearningbook.org>.
- [17] H. v. Hasselt, A. Guez, and D. Silver, “Deep reinforcement learning with double q-learning,” in *Proceedings of the Thirtieth AAAI Conference on Artificial Intelligence*, ser. AAAI’16. AAAI Press, 2016, p. 2094–2100.
- [18] P. J. Gmytrasiewicz and P. Doshi, “A framework for sequential planning in multi-agent settings,” *Journal of Artificial Intelligence Research*, vol. 24, p. 49–79, Jul. 2005. doi: 10.1613/jair.1579. [Online]. Available: <http://dx.doi.org/10.1613/jair.1579>
- [19] B. Ghojogh and A. Ghodsi, “Recurrent neural networks and long short-term memory networks: Tutorial and survey,” 2023. [Online]. Available: <https://arxiv.org/abs/2304.11461>
- [20] Z. C. Lipton, D. C. Kale, R. Wetzel *et al.*, “Modeling missing data in clinical time series with rnns,” *Machine Learning for Healthcare*, vol. 56, no. 56, pp. 253–270, 2016.
- [21] C. Szegedy, V. Vanhoucke, S. Ioffe, J. Shlens, and Z. Wojna, “Rethinking the inception architecture for computer vision,” in *2016 IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2016. doi: 10.1109/CVPR.2016.308 pp. 2818–2826.
- [22] S. Ross, G. Gordon, and D. Bagnell, “A reduction of imitation learning and structured prediction to no-regret online learning,” in *Proceedings of the Fourteenth International Conference on Artificial Intelligence and Statistics*, ser. Proceedings of Machine Learning Research, G. Gordon, D. Dunson, and M. Dudík, Eds., vol. 15. Fort Lauderdale, FL, USA: PMLR, 11–13 Apr 2011, pp. 627–635. [Online]. Available: <https://proceedings.mlr.press/v15/ross11a.html>
- [23] O. Bastani, Y. Pu, and A. Solar-Lezama, “Verifiable reinforcement learning via policy extraction,” in *Proceedings of the 32nd International Conference on Neural Information Processing Systems*, ser. NIPS’18. Red Hook, NY, USA: Curran Associates Inc., 2018, p. 2499–2509.
- [24] S. M. Lundberg and S.-I. Lee, “A unified approach to interpreting model predictions,” in *Proceedings of the 31st International Conference on Neural Information Processing Systems*, ser. NIPS’17. Red Hook, NY, USA: Curran Associates Inc., 2017. ISBN 9781510860964 p. 4768–4777.

- [25] Y. Wen, S. Li, R. Zuo, L. Yuan, H. Mao, and P. Liu, “Skilltree: explainable skill-based deep reinforcement learning for long-horizon control tasks,” in *Proceedings of the Thirty-Ninth AAAI Conference on Artificial Intelligence and Thirty-Seventh Conference on Innovative Applications of Artificial Intelligence and Fifteenth Symposium on Educational Advances in Artificial Intelligence*, ser. AAAI’25/IAAI’25/EAAI’25. AAAI Press, 2025. doi: 10.1609/aaai.v39i20.35451. ISBN 978-1-57735-897-8. [Online]. Available: <https://doi.org/10.1609/aaai.v39i20.35451>
- [26] I. Alsterfalk and A. Lindqvist, “Symbolic representation of knowledge for pursuit-evasion games,” 2024.
- [27] J. Sha, H. Shindo, Q. Delfosse, K. Kersting, and D. S. Dhami, “Expil: Explanatory predicate invention for learning in games,” 2024. [Online]. Available: <https://arxiv.org/abs/2406.06107>
- [28] A. M. Alaa and M. van der Schaar, *Demystifying black-box models with symbolic metamodels*. Red Hook, NY, USA: Curran Associates Inc., 2019.
- [29] A. Koul, S. Greydanus, and A. Fern, “Learning finite state representations of recurrent policy networks,” 2018. [Online]. Available: <https://arxiv.org/abs/1811.12530>

Appendix A

Supporting materials

The source code for the project is available on: [GitHub](#) at XAI-for-Knowledge-Based-Strategies-in-Grid-Based-PEG.

€€€€ For DIVA €€€€

```
{
  "Author1": { "Last name": "Arbman",
    "First name": "Oscar",
    "Local User Id": "oarbman",
    "E-mail": "oarbman@kth.se",
    "organisation": { "L1": "School of Electrical Engineering and Computer Science",
    }
  },
  "Cycle": "2",
  "Course code": "DA231X",
  "Credits": "30.0",
  "Degree1": { "Educational program": "Degree Programme in Computer Science and Engineering",
    "programcode": "CDATE",
    "Degree": "Degree of Master of Science in Engineering",
    "subjectArea": "Computer Science and Engineering"
  },
  "Title": {
    "Main title": "Explainable Artificial Intelligence for Knowledge-Based and Nested Belief-Based Strategies in Grid-Based Pursuit-Evasion Games",
    "Language": "eng" },
    "Alternative title": {
      "Main title": "Förklarbar Artificiell Intelligens för Kunskapsbaserade och Hierarkiska Trosbaserade Strategier i Rutnätsbaserade Jakt-Flykt Spel",
      "Language": "swe"
    },
  },
  "Supervisor1": { "Last name": "Guruv",
    "First name": "Dillian",
    "Local User Id": "dillian",
    "E-mail": "Dillian@kth.se",
    "organisation": { "L1": "School of Electrical Engineering and Computer Science",
      "L2": "Theoretical Computer Science" }
  },
  "Examiner1": { "Last name": "Dam",
    "First name": "Mads",
    "Local User Id": "mfd",
    "E-mail": "mfd@kth.se",
    "organisation": { "L1": "School of Electrical Engineering and Computer Science",
      "L2": "Theoretical Computer Science" }
  },
  "National Subject Categories": "10201",
  "SDGs": "12",
  "Other information": { "Year": "2026", "Number of pages": "xvi,67"},
  "Copyrightleft": "copyright",
  "Series": { "Title of series": "TRITA – XXX-EX", "No. in series": "2025:0000" },
  "Opponents": { "Name": "Christoforos Seas"},
  "Presentation": { "Date": "2026-06-04 14:00"
    },
  "Language": "Eng",
  "Room": "Via Zoom https://kth-se.zoom.us/j/68925280280",
  "Address": "Lindstedtsvägen 3, level 4",
  "City": "Stockholm",
  "Number of lang instances": "2",
  "Abstract[eng]": "€€€€
```

Explainable Artificial Intelligence (XAI) increases trust and transparency in AI systems, helping identify potential misalignments in a model's reasoning process. In this thesis, agents reason based on two types of knowledge that arise from the fact that agents have imperfect information about the state: first-order knowledge, which captures what an agent can infer about the current hidden state based on its own observation history; and second-order knowledge, which also accounts for deducible first-order knowledge of other agents. Previous work on knowledge-based strategy synthesis in grid-based pursuit-evasion games with imperfect information has not provided convincing evidence that Deep Q Network (DQN) based strategies that use second-order knowledge outperform those that use only first-order knowledge. This lack of evidence raises the question of whether agents actually benefit from and utilize second-order knowledge when making decisions in this environment. Consequently, a goal of the project is to develop a framework for extracting interpretable, knowledge-based strategies from DQNs to determine whether an agent with second-order knowledge employs them in its reasoning process within this environment.

The environment this thesis investigates is a (\$15 \times 15\$) grid-based pursuit-evasion game with two pursuers and one evader with imperfect information. However, because the current second-order knowledge-based update function produces too large representations, this thesis introduces a training pipeline to approximate second-order knowledge with a neural network. To distinguish it from the original knowledge-based update function, we refer to the approximation as the belief-based update model. The thesis then presents a symbolic representation that enables the use of an imitation learning method called INTERPRETER, which generates an interpretable decision-tree strategy in Python code that mimics the oracle DQN's actions. The thesis's three key findings were: firstly, that the decision tree strategy using second-order beliefs achieved the highest reward of any investigated strategy, with the lowest average steps to a terminal state of \$18.976\$, and a win rate of

\$99.991\%; and secondly, that the strategy bases its decisions partly on these second-order beliefs. Moreover, thirdly, an interpretable, completely formulaic from-observations-to-actions, first-order, knowledge-based strategy that was almost surely winning, losing only \$2\$ out of \$1,000,000\$ games, with an average number of steps to a terminal state of \$20.031\$, comparatively a naive perfect-information strategy, that always minimizes the distance to the evader achieved a \$15.742\$ average steps to a terminal state, with a \$100\%\$ win rate.

"Abstract[swe]": €€€€

Förklarbar artificiell intelligens (XAI) ökar förtroendet och förståelsen för AI-nätverk, vilket kan hjälpa till att identifiera potentiella fel i hur neurala nätverk resonerar. I denna avhandling resonerar agenter utifrån två typer av kunskap som uppstår eftersom agenterna har ofullständig information om spelets nuvarande tillstånd. Första ordningens kunskaper, fångar slutsatserna en agent kan dra om det nuvarande speltillståndet baserat på agentens observationshistoria och andra ordningens kunskaper, fångar också de härledbara första ordningens kunskaperna om de andra agenterna. Tidigare strategisyntes arbeten inom kunskapsbaserade förföljelse-undvikande spel på en rutnäts baserad miljö med ofullständig information har inte funnit övertygande bevis för att djupa Q-nätverks (DQN)-baserade strategier som har tillgång till andra ordningens kunskaper är bättre än de som endast har första ordningens kunskaper. Denna brist på bevis väcker frågan om agenter faktiskt drar nytta av och använder andra ordningens kunskaper när de fattar beslut i denna miljö. Därmed är ett mål med denna avhandling att utveckla ett ramverk för att extrahera tolkningsbara, kunskapsbaserade strategier från DQN för att avgöra om en agent med andra ordningens kunskaper använder dem i sina resonemang i denna miljö.

Miljön som denna avhandling undersöker är ett (\$15 \times 15\$) rutnätbaserat förföljande-undvikande-spel med \$2\$ förföljare och \$1\$ undvikare, där informationen är ofullständig. Eftersom den nuvarande andra ordningens kunskapsbaserade uppdateringsfunktionen producerar för stora representationer, introducerar denna avhandling en träningsprocess för att approximera andra ordningens kunskaper med ett neuralt nätverk. För att skilja denna approximation från den ursprungliga kunskapsbaserade uppdateringsfunktionen refererar vi till approximationen som den trosbaserade uppdaterings modellen. Avhandlingen presenterar sedan en symbolisk representation som möjliggör användningen av en imitation-inlärningsmetod som kallas INTERPRETER, vilket genererar en tolkningsbar beslutsträdstrategi i Python-kod som efterliknar orakel-DQNs handlingar. Avhandlingens tre viktigaste resultat var: för det första, att beslutsträdstrategin som använder andra ordningens trosuppfattningar uppnådde den högsta belöningen av alla undersökta strategier, med den lägsta genomsnittliga mängden steg till ett terminalt tillstånd på \$18,976\$ och en vinstfrekvens på \$99,991\%;\$ och för det andra, att strategin delvis baserar sina beslut på sina andra ordningens trosuppfattningar. Dessutom, för det tredje, hittades en komplett formell första ordningens kunskapsstrategi från observation till handling, som var nästan garanterat vinnande, som enbart förlorade \$2\$ av \$1,000,000\$ matcher, med ett genomsnittligt antal steg till ett terminalt tillstånd på \$20,031\$. Jämförelsevis hade en naiv avståndsförminskande strategi med perfekt information ett genomsnittligt antal steg till ett terminalt tillstånd på \$15,742\$ och en vinstfrekvens på \$100\%\$.

€€€€.

"Keywords[eng]": €€€€

Artificial Intelligence, Explainable Artificial Intelligence, Reinforcement Learning, Imperfect Information, Pursuit-Evasion Games, Grid-Based, Knowledge-Based Strategies, Deep Q-Networks, Convolutional Neural Networks, Long Short-Term Memory, Decision Trees, INTERPRETER, Knowledge-Based Update, Belief-Based Update €€€€.

€€€€.

"Keywords[swe]": €€€€

Artificiell Intelligens, Förklarbar Artificiell Intelligens, Förstärkningsinlärning, Imperfekt Information, Förföljelse-Undvikande Spel, Rutnäts Baserad, Kunskapsbaserade Strategier, Djupa Q-Nätverk, Falttningsnätverk, Långt Korttidsminne, Beslutsträd, INTERPRETER, Kunskapsbaserad Uppdatering, Trosbaserad Uppdatering €€€€, }



acronyms.tex

```
%%% Local Variables:
%%% mode: latex
%%% TeX-master: t
%%% End:
% The following command is used with glossaries-extra
\setabbreviationstyle[acronym]{long-short}
% The form of the entries in this file is \newacronym{label}{acronym}{phrase}
%                                     or \newacronym[options]{label}{acronym}{phrase}
% see "User Manual for glossaries.sty" for the details about the options, one example is shown below
% note the specification of the long form plural in the line below
%\newacronym[longplural={Debugging Information Entities}]{DIE}{DIE}{Debugging Information Entity}
%
% The following example also uses options
%\newacronym[shortplural={OSes}, firstplural={operating systems (OSes)}]{OS}{OS}{operating system}

% note the use of a non-breaking dash in long text for the following acronym
%\newacronym{IQL}{IQL}{Independent -QLearning}

% example of putting in a trademark on first expansion
%\newacronym[first={NVIDIA OpenSHMEM Library (NVSHMEM\texttrademark)}]{NVSHMEM}{NVSHMEM}{NVIDIA OpenSHMEM Library}

\newacronym{Adam}{Adam}{Adaptive Moment Estimation}
\newacronym{AI}{AI}{Artificial Intelligence}

\newacronym{BBU}{BBU}{Belief-Based Update}
\newacronym{BPTT}{BPTT}{Backpropagation Through Time}

\newacronym{CNN}{CNN}{Convolutional Neural Network}

\newacronym{DAGGER}{DAGGER}{Dataset Aggregation}

\newacronym{DNN}{DNN}{Deep Neural Network}

\newacronym{DRQN}{DRQN}{Deep Recurrent Q-Network}

\newacronym{DT}{DT}{Decision Tree }

\newacronym{DQN}{DQN}{Deep Q-Network}

\newacronym{INTERPRETER}{INTERPRETER}{INTERpretable Editable tRee Programs for ReinforcEmenT lEaRning}

\newacronym{KBU}{KBU}{Knowledge-Based Update}

\newacronym{LSTM}{LSTM}{Long Short-Term Memory}

\newacronym{MAGIIAN}{MAGIIAN}{Multi-Agent Game with Imperfect Information Against Nature}

\newacronym{MDP}{MDP}{Markov Decision Process}

\newacronym{MLP}{MLP}{Multilayer Perceptron}

\newacronym{MNM}{MNM}{Model Non-Manipulability}

\newacronym{MNO}{MNO}{Model Non-Observability}

\newacronym{PEG}{PEG}{Pursuit-Evasion Games}

\newacronym{PF-RNNs}{PF-RNNs}{Particle Filter Recurrent Neural Networks}

\newacronym{POMDP}{POMDP}{Partially Observable Markov Decision Process}

\newacronym{ReLU}{ReLU}{Rectified Linear Unit}

\newacronym{RL}{RL}{Reinforcement Learning}

\newacronym{RNN}{RNN}{Recurrent Neural Network}

\newacronym{TD}{TD}{Temporal Difference}

\newacronym{XAI}{XAI}{Explainable Artificial Intelligence}
```